

An Introduction to CEDS in Transactional Systems

*Created by Michigan's Center for
Educational Performance and Information
and shared with the
CEDS Open Source Community*



Last Updated 10/1/2026

Table of Contents



1. Introduction

- A. [Where does this training fit in the sequence of trainings?](#)
- B. [Who is this training for? What level is this training?](#)
- C. [What are the goals of this training? Must I read it all?](#)

2. [CEDS Integrated Data Store \(IDS\)](#)

- A. [What is an IDS? How does normalization affect work in the IDS?](#)
- B. [Case Study #1 – How are Identifiers stored in the IDS?](#)
- C. [Case Study #2 – Everything is an Organization](#)
- D. [How are record management fields used in the IDS?](#)

3. [CEDS Ontology](#)

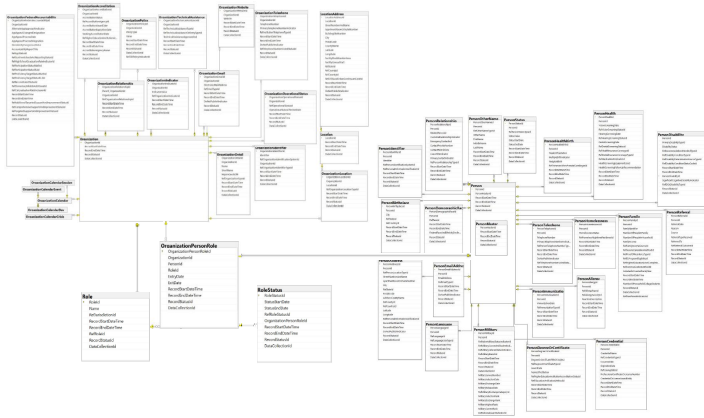
- A. [What is an ontology? Triples? A knowledge graph?](#)
- B. [How is the CEDS ontology similar to and different from the IDS?](#)
- C. [Codes, namespaces, and ontology formatting](#)

4. [JavaScript Object Notation for Linked Data \(JSON-LD\)](#)

- A. [What is JSON-LD and why would we use it?](#)
- B. [How does JSON-LD compare to the IDS?](#)
- C. [How can the CEDS Ontology be implemented in JSON-LD?](#)
- D. [How do context files help make JSON-LD more human-readable?](#)

5. [Additional Resources](#)

Where does this training fit in the sequence of trainings?



Transactional Layer (IDS or JSON-LD)

Designed for transactional data – normalized structures make writing and updating application data faster.



Data Warehouse (DW)

Designed for storing longitudinal data – the star schema makes querying to read data faster.

<Link to CEDS Data Warehouse training module coming soon!>



Semantic Layer

Designed for humans. Datasets and views are optimized for specific purposes and tools.

<Link to CEDS Semantic Layer training module coming soon! >



Who is this training for?

What level is it?

This training is designed for those who are responsible for any of the following:

- Developing business rules for data movement, data quality checks, or reporting
- Validating data that has been submitted and/or ETL'd
- Producing recurring and/or ad hoc reports using data that passed through the transactional layer.

This training assumes:

- That you have some familiarity with CEDS. (If you are new to CEDS, start [here](#) instead.)
- That most of your work will be done using the Semantic Layer or CEDS Data Warehouse, but you'd like to learn about the Transactional Layer.



NOTE: This training does *not* cover how to write the code to create JSON-LD. If you are interested in learning more about the technical aspects of JSON-LD, there are links to more technical trainings at the end of this PowerPoint.

Training Goals



After completing this training, you should be able to:

- Describe the benefits of using the CEDS IDS, the CEDS Ontology, and JSON-LD.
- Understand some of the most common structures and conventions associated with the CEDS IDS, the CEDS Ontology, and JSON-LD.
- Think about how the structures and conventions of the CEDS IDS, the CEDS Ontology, and JSON-LD might impact your work as an analyst.

Q: My organization only uses the IDS (or only uses JSON-LD). Can I skip the other sections?

If your organization only uses:	Review these sections:
CEDS IDS	CEDS IDS and CEDS Ontology*
JSON-LD	CEDS IDS*, CEDS Ontology, and JSON-LD

* Why can't I skip directly to my section(s)?

If you are only using JSON-LD, you still need to read the IDS section because the Ontology was based on the IDS, and the Ontology section of this training builds on foundational concepts explained in the IDS section.

If you only use the IDS, you'll still want to read the CEDS Ontology section since the Ontology will guide future changes in the IDS. The JSON-LD section is optional to IDS users but may still be worth reviewing since JSON-LD is an increasingly common way to represent data.

**Q: This is a lengthy slide deck
– am I expected to read it all in one sitting?**

No.

There are distinct section breaks between the IDS, Ontology and JSON-LD chapters of this training. You are encouraged to pause and reflect on what you've read and learned.

Additionally, each chapter is further broken into sections (as you can see in the [Table of Contents](#)) so that each person can 'chunk' the learning into pieces that best fit their schedule.

The CEDS Integrated Data Store (IDS)

What is the Integrated Data Store (IDS)?

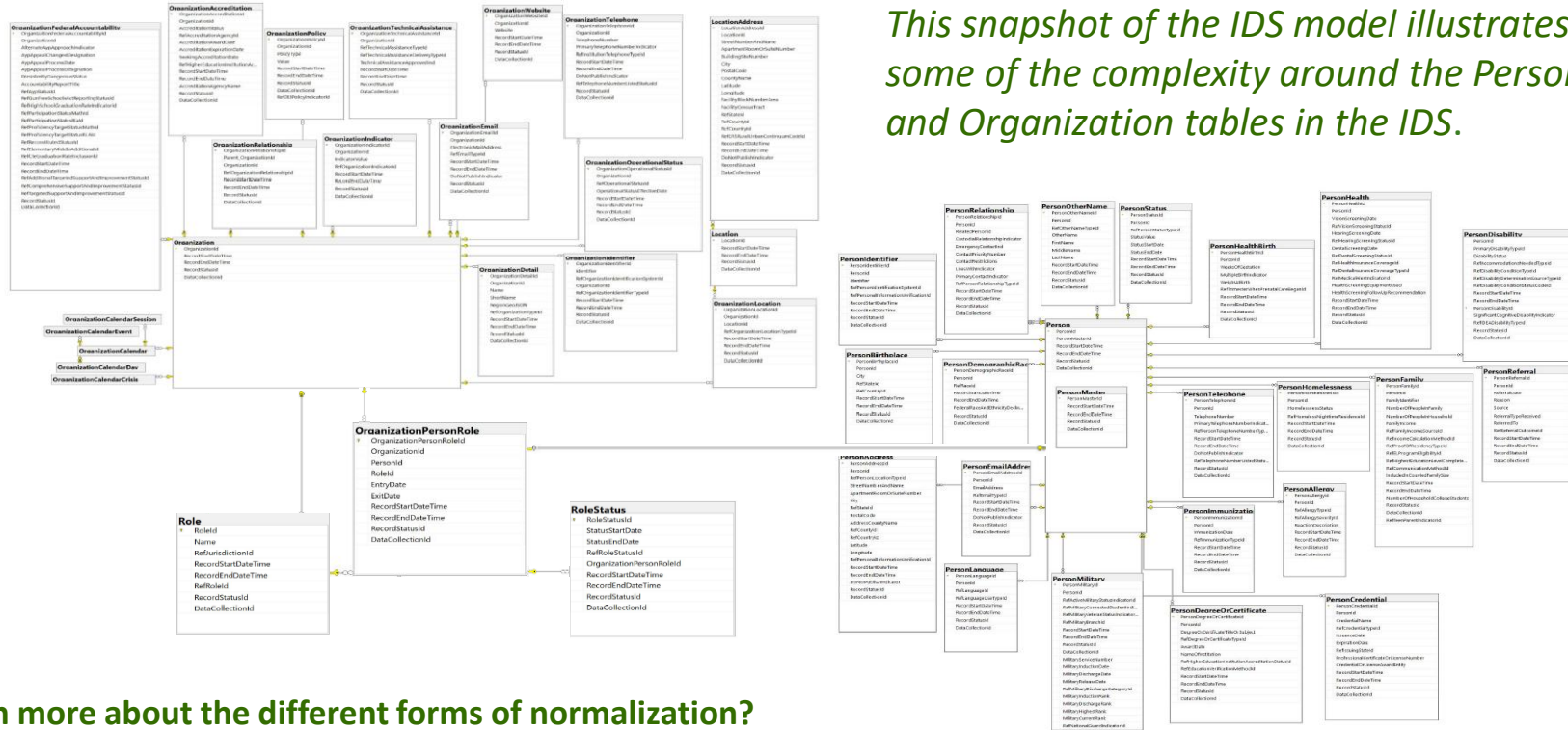
- Transactional data are bits of information captured when an event happens in an application. For example, updating a person's last name is an event. Transactional data captured during that event likely includes more than just their new last name – it could include the date and time of when the 'LastName' field was updated as well as the person or organization that submitted the change.
- The Integrated Data Store (IDS) is based on CEDS but the syntax and table structures have been optimized for transactional data, enabling applications to write/update data to the IDS faster. This makes the IDS useful for near real-time reporting about educational data.



"The Common Education Data Standards (CEDS) Integrated Data Store (IDS) factors the complete set of CEDS elements, entities and attributes with standard technical syntax and [3rd normal form database normalization](https://github.com/CEDStandards/CEDS-IDS/tree/master/doc). The IDS Model provides a standard framework for integration of P-20W data systems through a well-normalized operational data store. It can be used for transactional education data systems, to perform near real-time reporting, and as a standard staging area to load the CEDS Data Warehouse. The CEDS Data Model Guide can be found here <https://github.com/CEDStandards/CEDS-IDS/tree/master/doc>."

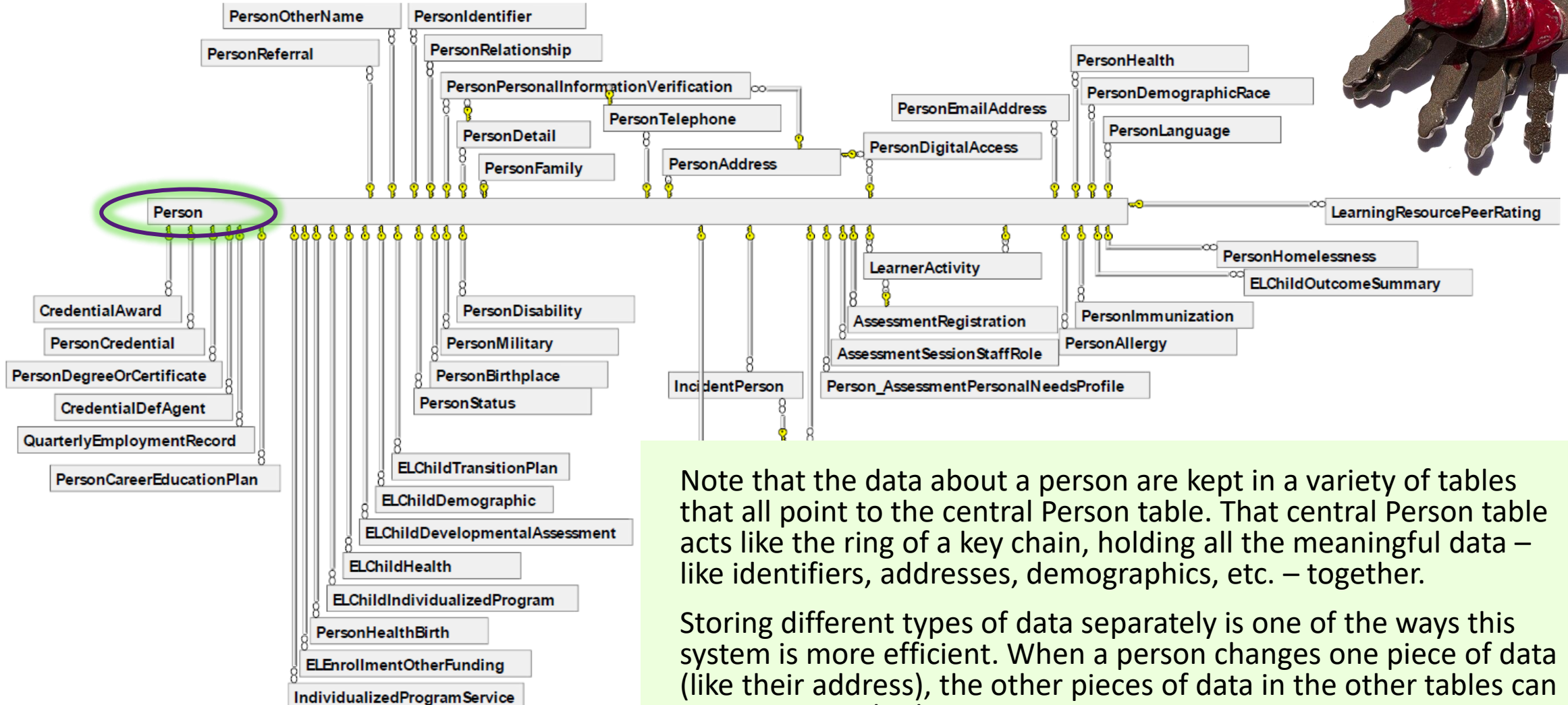
How does normalization affect work in the IDS?

Database normalization reduces redundancy in the data by breaking off discrete units of data to separate tables and removing certain types of dependencies between columns*. This improves consistency in the data and reduces storage costs. However, the quantity of foreign keys needed to achieve this can make the IDS more challenging for human analysts to navigate. **The CEDS IDS has over 1,000 tables**



Want to learn more about the different forms of normalization?
See [Additional Technical Resources](#) at the end of this document

The keychain analogy



Note that the data about a person are kept in a variety of tables that all point to the central Person table. That central Person table acts like the ring of a key chain, holding all the meaningful data – like identifiers, addresses, demographics, etc. – together.

Storing different types of data separately is one of the ways this system is more efficient. When a person changes one piece of data (like their address), the other pieces of data in the other tables can remain untouched.

A few IDS conventions before we go on...



- **The primary system key for each table will be the name of the table with an 'Id' at the end.**
 - For example, **PersonId** is the key for the Person table, and **PersonIdentifierId** is the key to the PersonIdentifier table. These are **system-generated keys used to link tables**. They are NOT used to store meaningful identifiers assigned by an organization – we'll talk about those on the next slide!
- **Tables that begin with 'Ref' are look-up tables that hold a specific option set.**
 - Option sets have a human-friendly **Description** (standard capitalization, spacing, and spelling), a detailed **Definition**, and a **Code** (no spaces, no special characters, software-friendly).
 - Some examples from the RefSpecialEducationAgeGroupTaught option set:

Option Code	Option Description	Option Definition
3TO5NOTK	3 through 5 (not in kindergarten)	The age range of special education students taught is 3 through 5 (not in kindergarten)
AGE5KTO21	5 (in kindergarten) through 21	The age range of special education students taught is 5 (in kindergarten) through 21

Note: The Option Code is unique within a given option set, but it is *not* globally unique. A code may be reused in multiple option sets. For example, the code 'State' is used to mean a 'State-assigned' value in multiple option sets. We'll talk about globally unique option codes in the CEDS Ontology section of this training.

Case Study #1: How are Identifiers stored in the IDS?



Most people have a variety of identifiers assigned to them throughout their lifetime. In CEDS, each identifier assigned to a person or organization is labeled with a 'Type' and an 'Identification System'.

The Person Identifier Type reflects **the person's role** at the time the identifier was assigned. For example, the identifier for a teacher at school would be a 'Staff Member Identifier' whereas the identifier for a student would be a 'Student Identifier'.

The Person Identification System documents **which kind of organization assigned the identifier**, thereby allowing us to store (and distinguish) between multiple student identifiers, person identifiers, etc. For instance, a student might have one identifier assigned by their school district and a different identifier assigned by the college where they are dual-enrolled.

Person Identifier Type	Person Identification System	Person Identifier
Student Identifier	District-assigned number	12345
Student Identifier	State-assigned number	6677889900
Staff Member Identifier	State-assigned number	987654
Person Identifier	Social Security Administration number	123-45-6789

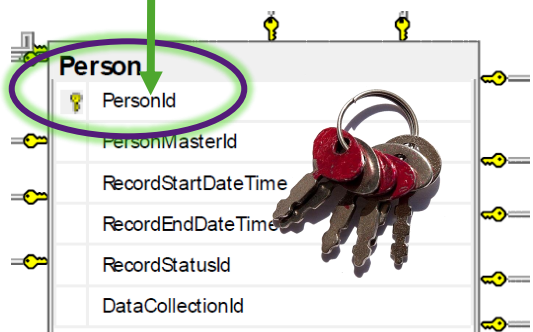
How is this implemented in the IDs tables?

PersonIdentifier

PersonIdentifierId
PersonId
Identifier
RefPersonIdentificationSystemId
RefPersonalInformationVerificationId
RecordStartDateTime
RecordEndDateTime
RecordStatusId
DataCollectionId

Every Identifier is stored in one table – PersonIdentifier – along with foreign keys to the relevant Person and to an option within the RefPersonIdentificationSystem.

The RefPersonIdentificationSystem option then has a foreign key to the relevant record in the RefPersonIdentifierType table.



Person
PersonId
PersonMasterId
RecordStartDateTime
RecordEndDateTime
RecordStatusId
DataCollectionId

RefPersonIdentificationSystem

RefPersonIdentificationSystemId
Description
Code
Definition
RefJurisdictionId
RefPersonIdentifierTypeId
SortOrder
RecordStartDateTime
RecordEndDateTime

RefPersonIdentifierType

RefPersonIdentifierTypeId
Description
Code
Definition
RefJurisdictionId
SortOrder
RecordStartDateTime
RecordEndDateTime

Person table FAQ

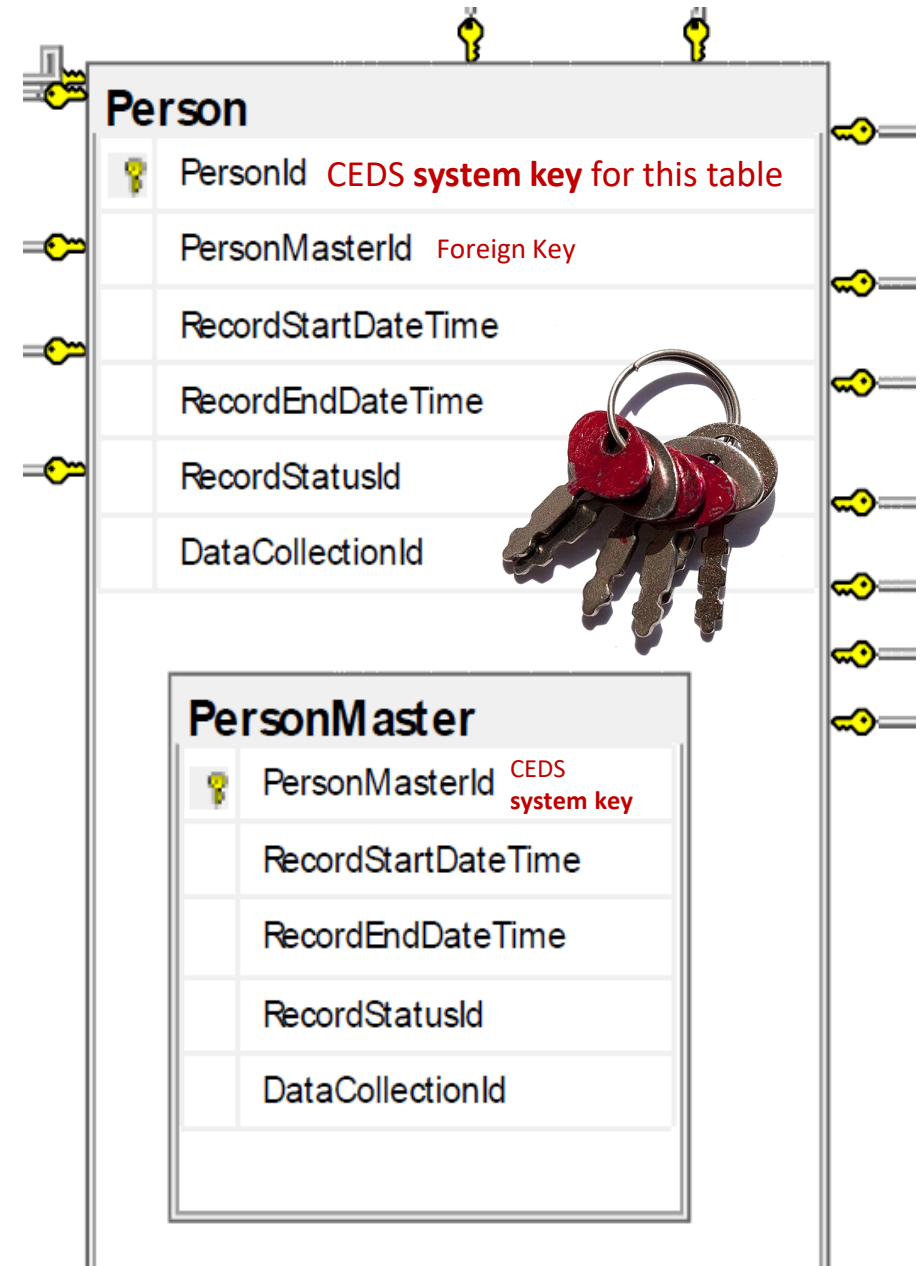
Q: Is any fact data stored in the Person table?

A: No. The Person table is only the ring of a keychain – or the table that links together all the data stored in other tables. For example, all identifiers are stored in the PersonIdentifier table and then each PersonIdentifier record points to the one Person record that they all apply to. This structure allows a 1-to-many relationship between a person and different data elements like identifiers, addresses, telephone numbers, etc.

Q: What is the PersonMaster table?

A: Ideally each individual only has one record in the Person table. But what if an individual has two records in the Person table? That's where PersonMaster becomes important.

PersonMaster is "A table used to [link] Person records that represent the same person but may be from different sources and may reflect differences in details."



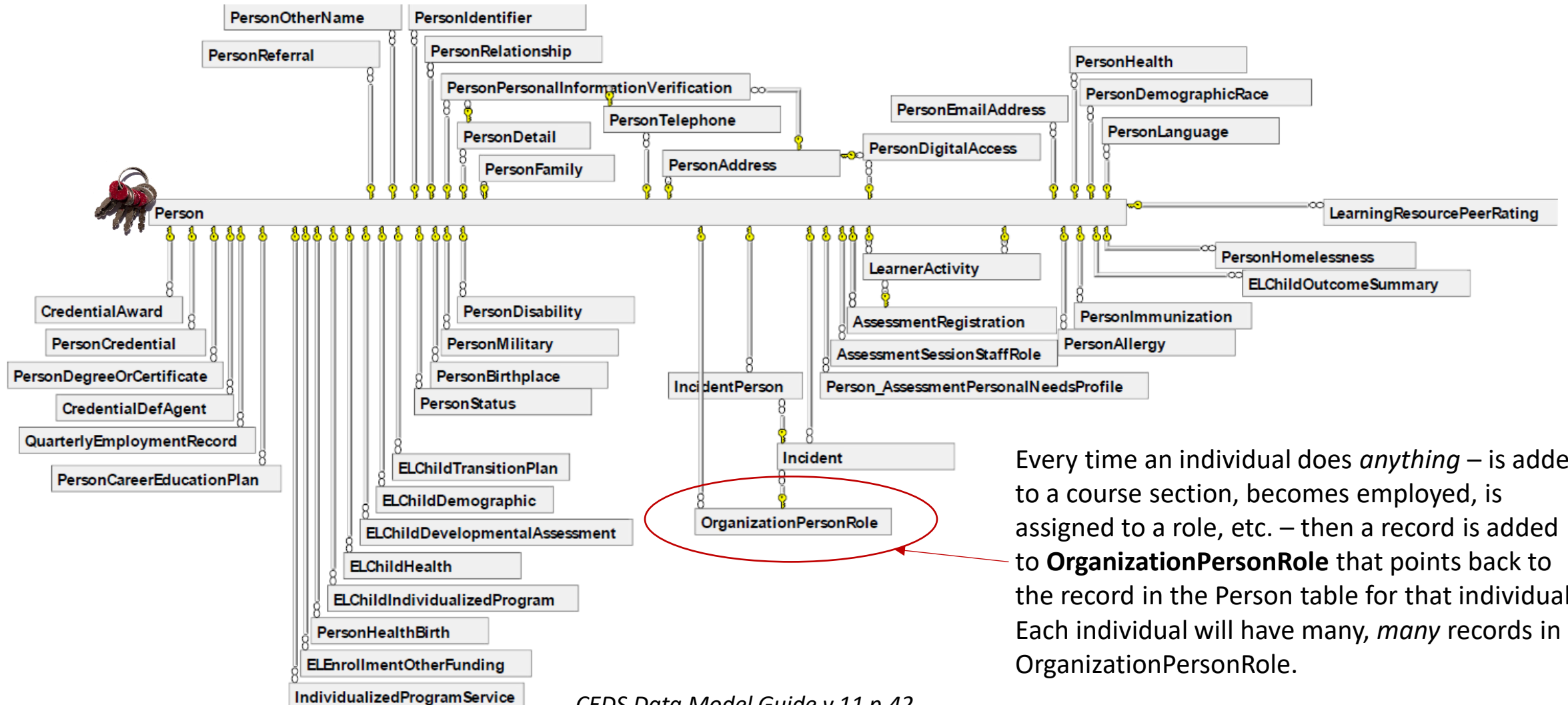
Person-linking in the IDS

The next three slides go into more detail about how person-linking works in the IDS.

If that is not relevant to your work, feel free to skip to Case Study #2

Many tables link to the Person table

This diagram shows the tables that link to the Person table, but it cannot show the scale of how *many* records will point to the Person table. Each person's record in the Person table will have dozens if not hundreds of records in the other tables all pointing to it.



Records in **PersonIdentifier** Table

PersonIdentifierId = 3
PersonId = 2
Identifier = 123456
RefPersonIdentificationSystemId
→ Points to **SEA-assigned Staff** Identifier
RecordStartDateTime 2018-05-04
RecordEndDateTime <null>
RecordStatusId
→ Points to **status & whoever set that status.**

PersonIdentifierId = 4
PersonId = 2
Identifier = 987654
RefPersonIdentificationSystemId
→ Points to the **District-assigned Staff** Identifier
RecordStartDateTime 2019-10-03
RecordEndDateTime <null>
RecordStatusId
→ Points to **status & whoever set that status.**

PersonIdentifierId = 5
PersonId = 2
Identifier = 111111
RefPersonIdentificationSystemId
→ Points to **SEA-assigned Person** Identifier
RecordStartDateTime 2025-12-03
RecordEndDateTime <null>
RecordStatusId
→ Points to **status & whoever set that status.**

Record in **Person** Table

PersonId = 2
PersonMasterId = 1
RecordStartDateTime 2018-05-04
RecordEndDateTime <null>
RecordStatusId
→ Points to **status & whoever set that status.**

Record in **PersonMaster** Table

PersonMasterId = 1
RecordStartDateTime 2018-05-04
RecordEndDateTime <null>
RecordStatusId
→ Points to **status & whoever set that status.**

This is an example of the ideal situation.

One individual has only **one** record in the Person Table.

Note that an individual can have multiple identifiers in the PersonIdentifier Table. NONE of those identifiers is same as the PersonId field because PersonId is the system-generated key for the Person Table.

Also remember that there will be likely dozens or hundreds of records in the fact and bridge tables that will use the PersonId to point to this distinct person. The system-generated PersonId key is what holds together all the data about this person.

NOTE: CEDS allows organizations to choose if every person will have a record in the PersonMaster table (easier to write clean code) or it is only be used for cases of merging/splitting (uses less space).

Dozens or hundreds of records in fact or bridge tables

PersonIdentifierId = 3
PersonId = 2
Identifier = 123456
RefPersonIdentificationSystemId
→ Points to SEA-assigned Staff Identifier
RecordStartDateTime 2018-05-04
RecordEndDateTime <null>
RecordStatusId → Points to who created record & its status

PersonIdentifierId = 4
PersonId = 2
Identifier = 987654
RefPersonIdentificationSystemId
→ Points to **District-assigned Staff** Identifier
RecordStartDateTime 2021-10-03
RecordEndDateTime <null>
RecordStatusId → Points to who created record & its status

*Dozens or hundreds of records in **fact or join tables***

PersonIdentifierId = 22
PersonId = 21
Identifier = 456789
RefPersonIdentificationSystemId
→ Points to **SEA-assigned Person** Identifier
RecordStartDateTime 2020-06-14
RecordEndDateTime 2023-01-12
RecordStatusId → Points to who created record & its status

*Dozens or hundreds of records in **fact or join tables***

Current Person Record

PersonId = 2
PersonMasterId = 1
RecordStartDateTime 2018-05-04
RecordEndDateTime <null>
RecordStatusId
→ Points to who created record & its status

PersonMasterId = 1
RecordStartDateTime 2018-05-04
RecordEndDateTime <null>
RecordStatusId
→ Points to who created record & its status

What happens when one person is discovered to have two records in the Person table?

Both Person records are set to point to the same PersonMaster. This allows the dozens (or hundreds) of fact and join table records that are associated with each Person record to now be recognized as belonging to the same individual.

There will be some duplicate records that will need to be 'closed' (i.e. have the **RecordEndDateTime set**) such as the duplicate PersonId record and any Person-specific records that should not have duplicates (e.g. duplicate PIC)

Closed Person Record

PersonId = 21
PersonMasterId = 1
RecordStartDateTime 2020-06-14
RecordEndDateTime 2023-01-12

Case Study #2: Everything is an ORGANIZATION

In CEDS, an Organization is defined as "An organized group of one or more people with particular purpose."

K12 Schools, Local Education Agencies and Employers are easy to recognize as 'organizations' under this definition. But Programs, Course Sections, and Facilities are also 'organizations' in CEDS.

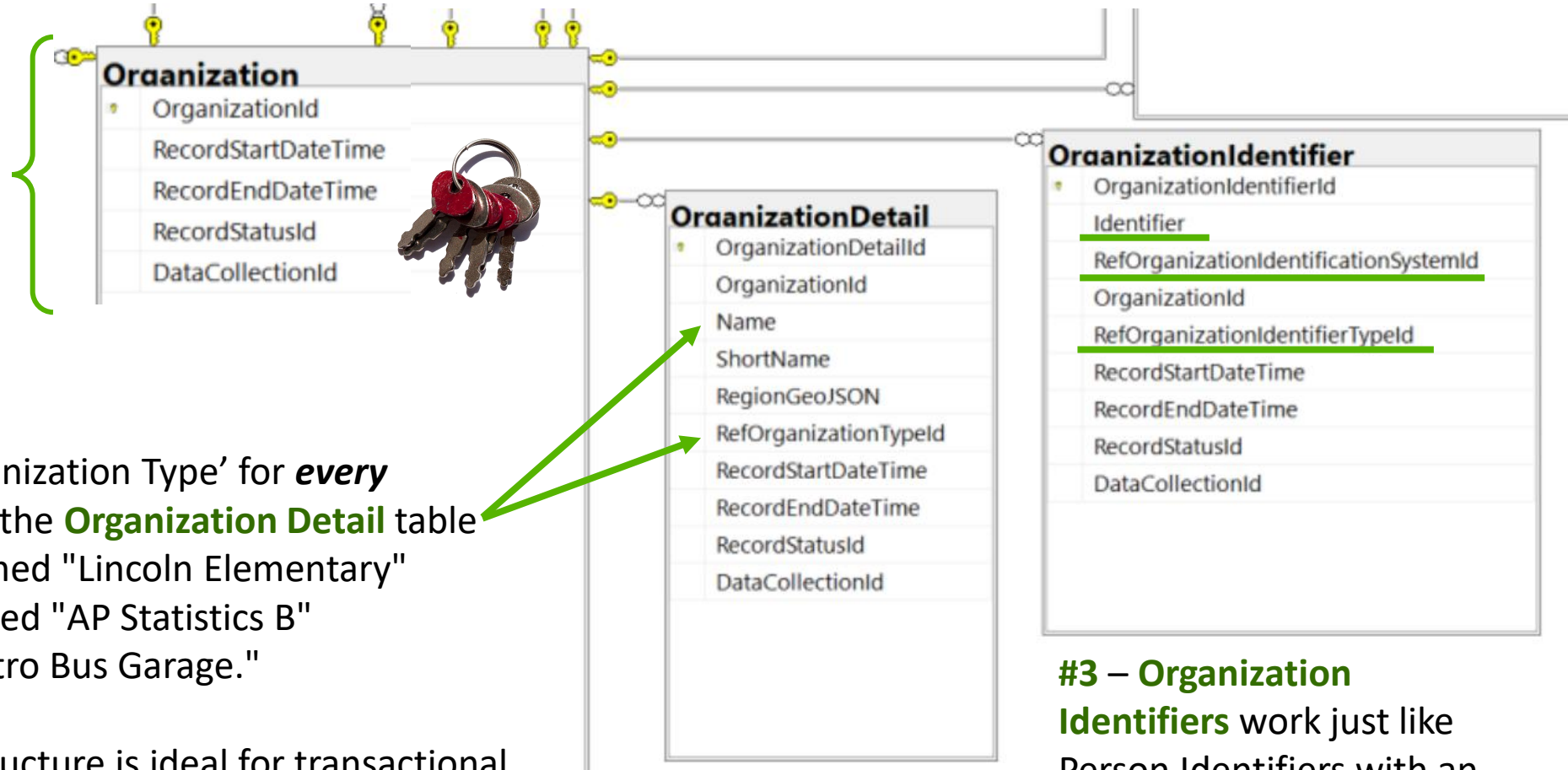
There are 32 different organization types in Version 14 of CEDS – leading some to joke that '*Everything* is an organization'



Accrediting Organization	Facility
Adult Education Provider	Governing Board
Affiliated Institution	IDEA Part C Lead Agency
Campus	Intermediate Educational Unit (IEU)
Charter School Authorizing Organization	K12 School
Charter School Management Organization	Local Education Agency (LEA)
Course	Postsecondary Institution
Course Section	Postsecondary System
Credentialing Organization	Professional Membership Organization
Early College	Program
Education Institution	Recruiter
Education Organization Network	Region
Emergency Response Agency	State Education Agency (SEA)
Employee Benefit Carrier	Service Provider
Employee Benefit Contributor	SHEEO Agency
Employer	Staff Development Provider

Organization data are normalized just like Person

#1 – Notice that the 'Organization' table works just like the 'Person' table – acting like a keychain ring linking together all the tables holding organization-related data.



#2 – The 'Name' and 'Organization Type' for *every* Organization are stored in the **Organization Detail** table – from the K12 School named "Lincoln Elementary" to the Course Section named "AP Statistics B" or the Facility named "Metro Bus Garage."

This kind of normalized structure is ideal for transactional systems which can quickly distinguish among these very different organizations using the **RefOrganizationType** table.

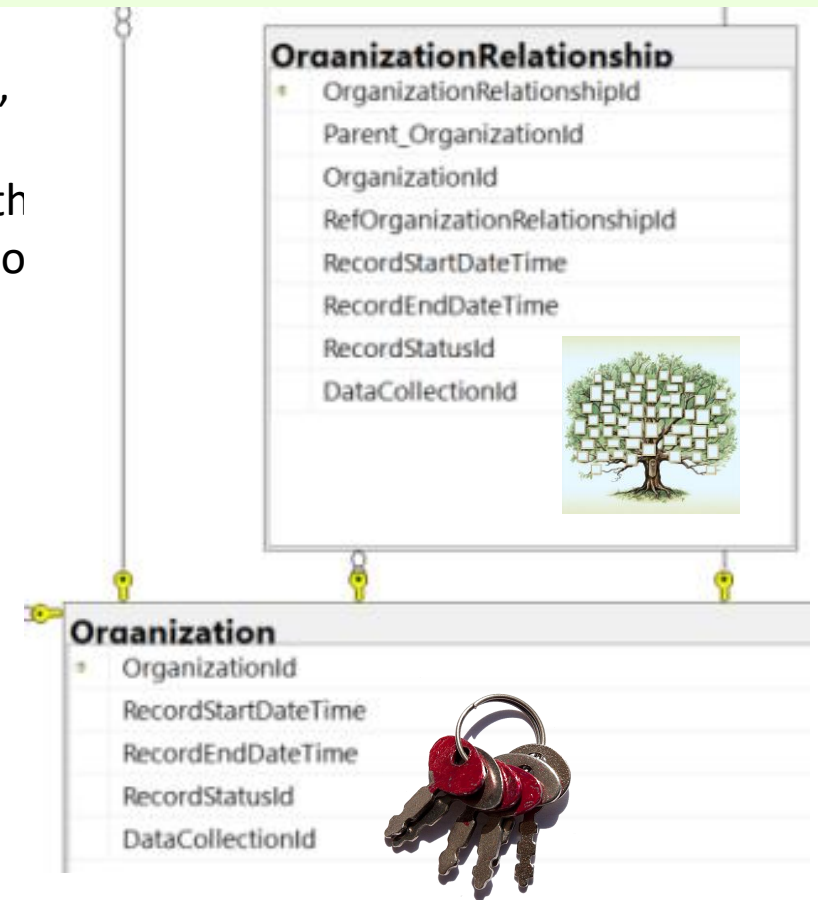
#3 – **Organization Identifiers** work just like Person Identifiers with an **Organization Identification System** and **Organization Identifier Type**.

Organization Relationship ≈ Family Tree

Most organizations have relationships with other organizations. For example, in Michigan, a public K12 School is typically owned and operated by a school district (a.k.a. Local Education Agency or LEA) which works in partnership with a regional agency or intermediate school district (a.k.a. Intermediate Education Unit or IEU). It may also be affiliated with a college or other organizations.

The 'Organization Relationship' table is responsible for documenting each relationship and the Organization Relationship Type. For example:

OrganizationId points to	Parent_OrganizationId points to	RefOrganizationRelationshipId points to the type of relationship
West Elementary	Smalltown Public Schools	Operating Body
West Elementary	Hamilton College	Related Body
Smalltown Public Schools	Hamilton IEU	Administrative Body (proposed for v.15)



Note: Parent_OrganizationId points to **the organization that is the object of the relationship**. Even though the fieldname has 'Parent' in it, that isn't necessarily the type of relationship. We instead look to RefOrganizationRelationship to identify **which kind of relationship this is**.

Tree image used with permission from:
<https://pixabay.com/illustrations/family-tree-family-life-cycle-wood-8249245/>

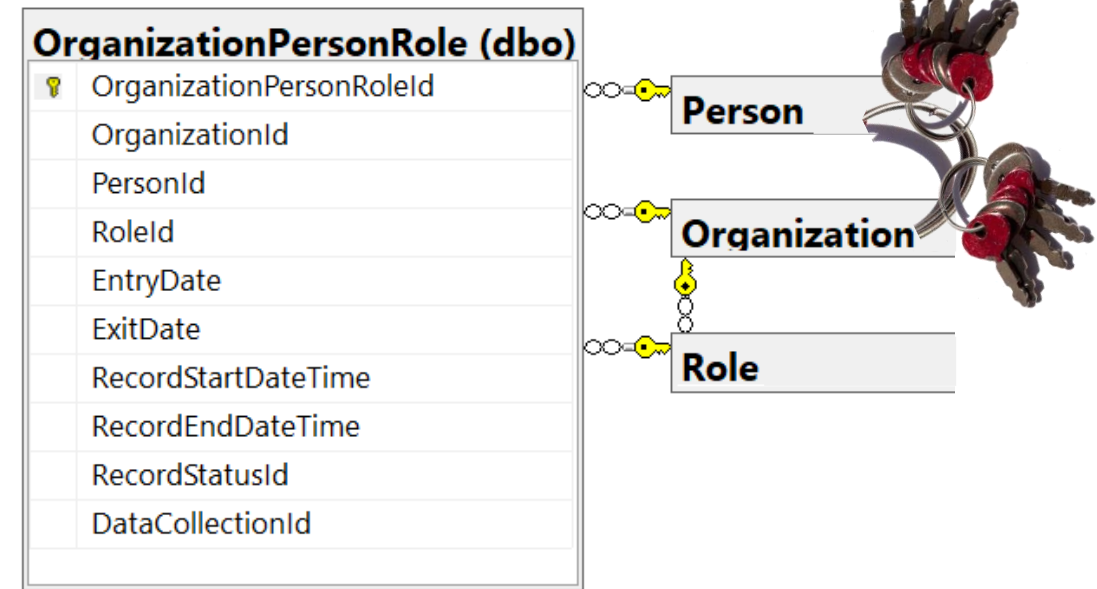
OrganizationPersonRole

– the keychain of keychains!

The Person table holds together all of person data.

The Organization table holds together organization data.

OrganizationPersonRole is where those two key chains get linked together based on the role that a person has at a *specific* organization during a *specific* range of time.

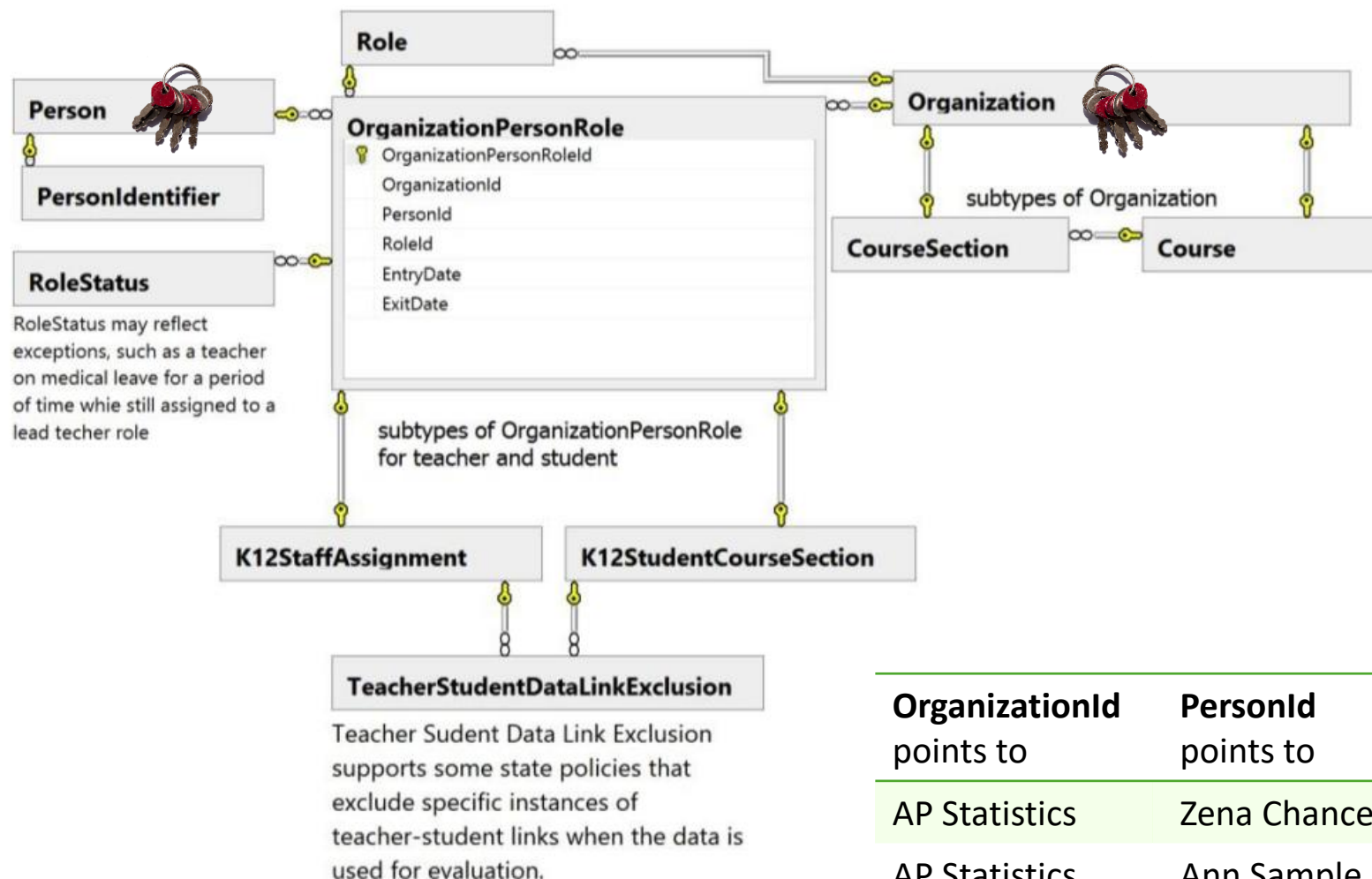


OrganizationId points to	PersonId points to	RoleId points to	Entry Date	Exit Date
East Middle School	Ann Sample	K12Student	2017-09-05	2020-06-12
East High School	Ann Sample	K12Student	2020-09-01	2025-05-28
AP Statistics	Ann Sample	K12Student	2023-09-05	2024-06-07
Hamilton College	Ann Sample	PS Student	2024-08-01	
East High School Athletics Dept.	Ann Sample	K12Staff	2025-11-15	

For example, Ann was a student at East Middle School and then at East High School.

While at East High School, she took AP Statistics. After that, she dual-enrolled at Hamilton College and then continued with her postsecondary studies there after graduation (so no exit date yet!).

Ann also got hired by her old high school's athletic **program** the fall after graduation.



Class rosters

Remember that a Course Section is a type of Organization. The OrganizationPersonRole table will hold the list of all people associated with each course section and their role as staff or student – effectively storing the class roster. Entry and Exit dates are specific to *this* role at *this* organization.

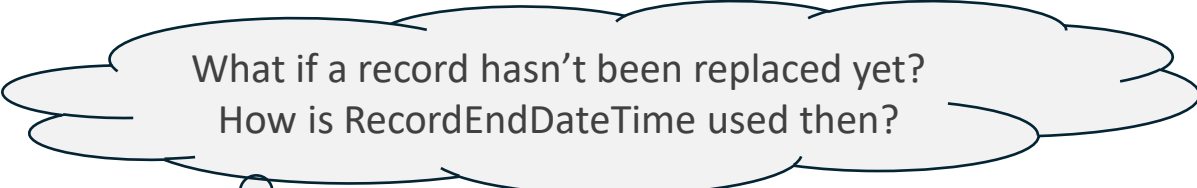
For example, Dan joined this course section mid-year and Bea left mid-year. If we want to know whether they changed schools, we'd look for the record where the OrganizationId points to the school instead of to a course section.

OrganizationId points to	PersonId points to	RoleId points to	Entry Date	Exit Date
AP Statistics	Zena Chance	K12Staff	2023-07-01	2024-06-30
AP Statistics	Ann Sample	K12Student	2023-09-05	2024-06-07
AP Statistics	Bea Example	K12Student	2023-09-05	2024-01-07
AP Statistics	CeeCee Instance	K12Student	2023-09-05	2024-06-07
AP Statistics	Dan Figure	K12Student	2023-10-05	2024-06-07

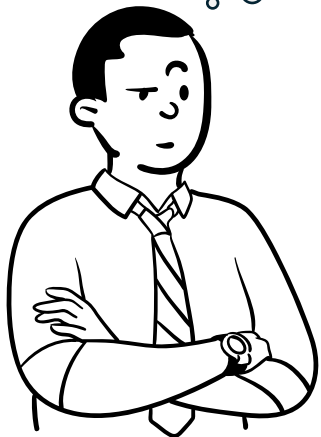
How are record management fields used?

Fact records will have

- **RecordStartDateTime** – when a record was created
- **RecordEndDateTime** – when a record was replaced with a more updated record



What if a record hasn't been replaced yet?
How is RecordEndDateTime used then?



For current records, many organizations choose to populate RecordEndDateTime with a date in the extreme distant future. For example, 9999-12-31.

Other organizations choose to leave it null, but this may impact performance.

- **DataCollectionId** – a foreign key to the DataCollection table enabling you to store details about when and how this data was collected.
- **RecordStatusId** – a foreign key to the RecordStatus table which can be used to track
 - The status of the data – whether it is in-progress, submitted, certified, etc.
 - The date the status was updated
 - the organization that committed the data or the person who created it

Time to Reflect & Digest on the IDS



- What are some of the benefits of using the CEDS IDS? Slides 9-10
- What are some of the common structures and conventions used in the IDS?
 - How are identifiers stored? Slides 13-15
 - What counts as an organization? Slide 20
 - How can we store a hierarchy of organizations? Slide 22
 - What kind of data are stored in the Person and Organization tables? Slides 11, 15 & 21
 - How is the PersonOrganizationRole table used? Slides 23-24
 - What are some naming conventions that are useful to understand? Slide 12
 - What kind of record management fields are used in the IDS? Slide 25
- REFLECT:
 - How might these structures and conventions impact your work?
 - What questions do you have?

NEXT UP: The CEDS Ontology!

The CEDS Ontology

What is the CEDS Ontology?

What is an 'ontology'?

The purpose of an ontology "is to describe things and the relationships that exist between them. From this machine-readable understanding, additional resources can be created such as JSON, JSON-LD, XML, and a host of other machine-readable standards. The CEDS Ontology makes it easier to link CEDS to other ontologies and standards through the use of equivalencies (an indication when one standard's element is equivalent to another standard's element) making translation a much more intuitive process and, again, machine-readable." ([CEDS GitBook | Ontology](#) 5/5/2026)



TLDR:

An ontology is a formal, written document that clearly describes shared concepts.

- Data elements are defined.
- Relationships between data elements are also defined.
- The definitions of elements and relationships are collectively agreed upon.
- It's structured to be easy for machines to read.

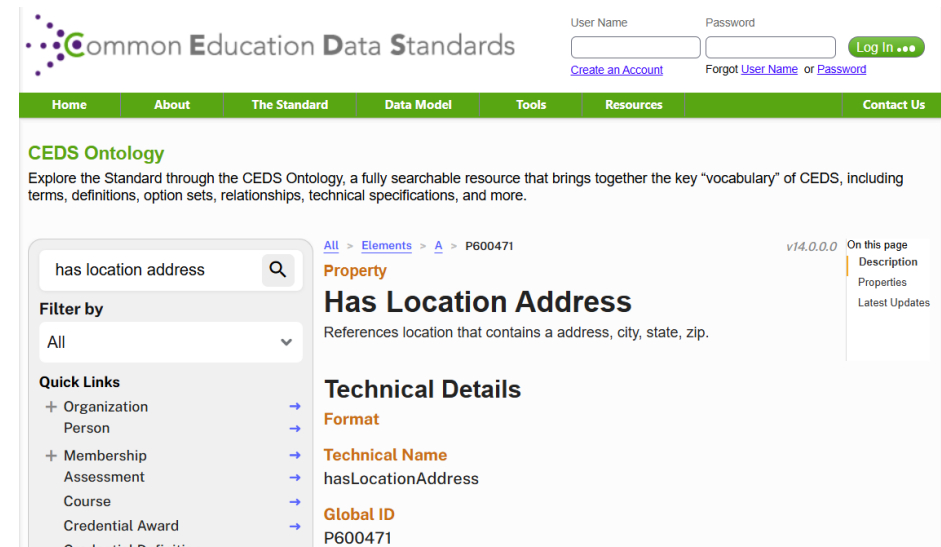
Does the CEDS Ontology replace the IDS?

No - "The CEDS Ontology is a new way of looking at CEDS... The CEDS Ontology **builds** on the CEDS Integrated Data Store model by not only demonstrating relationships exist between elements through Primary and Foreign Key relationships, but by also providing definitions and meaning about those relationships through Object Properties."

(<https://github.com/CEDStandards/CEDS-Ontology> ReadMe, 3/29/2025)

As of November 2025, **CEDS stores all their data definitions for their elements – and the descriptions of the relationships between those elements – in the CEDS Ontology** which you can access via their website or by downloading the machine-readable document.

Ontology Website



Ontology Document

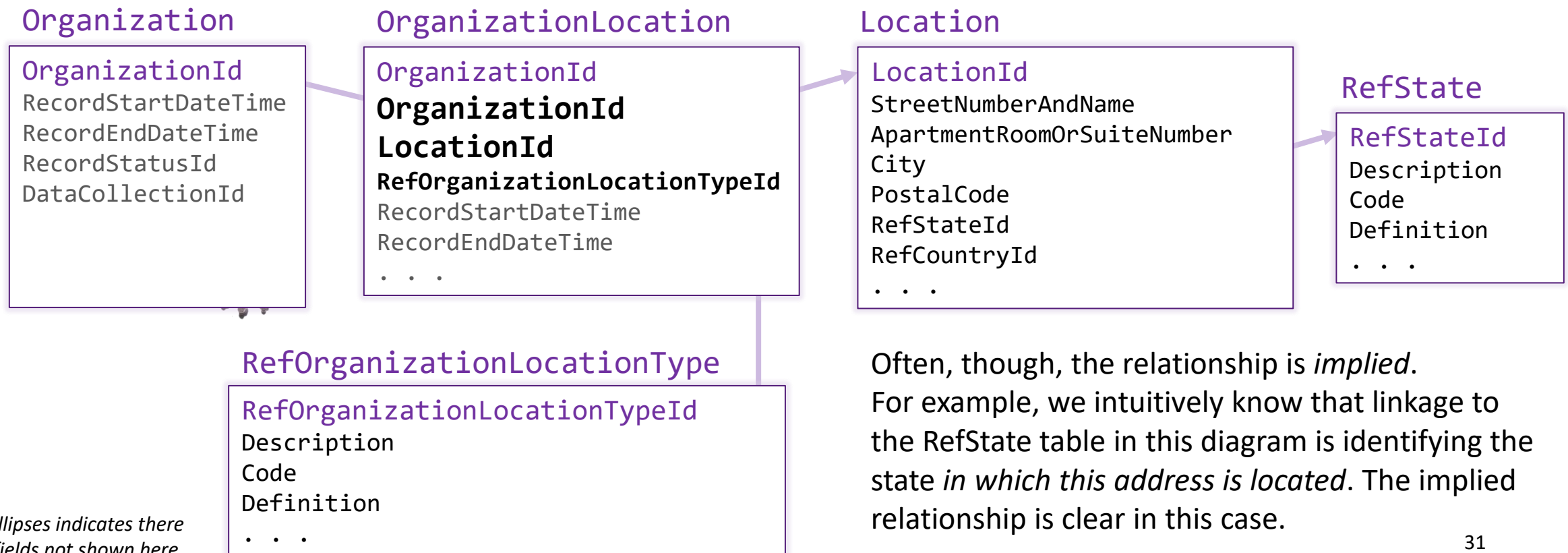
```
</rdf:Property>
<rdf:Property rdf:about="http://ceds.ed.gov/terms#P600471">
  <dc:creator>Common Education Data Standards</dc:creator>
  <dc:identifier rdf:datatype="http://www.w3.org/2001/XMLSchema"
  <rdfs:label>Has Location Address</rdfs:label>
  <schema:domainIncludes rdf:resource="http://ceds.ed.gov/ter
  <schema:domainIncludes rdf:resource="http://ceds.ed.gov/ter
  <schema:domainIncludes rdf:resource="http://ceds.ed.gov/ter
  <schema:domainIncludes rdf:resource="http://ceds.ed.gov/ter
  <schema:rangeIncludes rdf:resource="http://ceds.ed.gov/term
  <skos:notation>hasLocationAddress</skos:notation>
</rdf:Property>
```

What does it mean to say an ontology "defines relationships through object properties"?



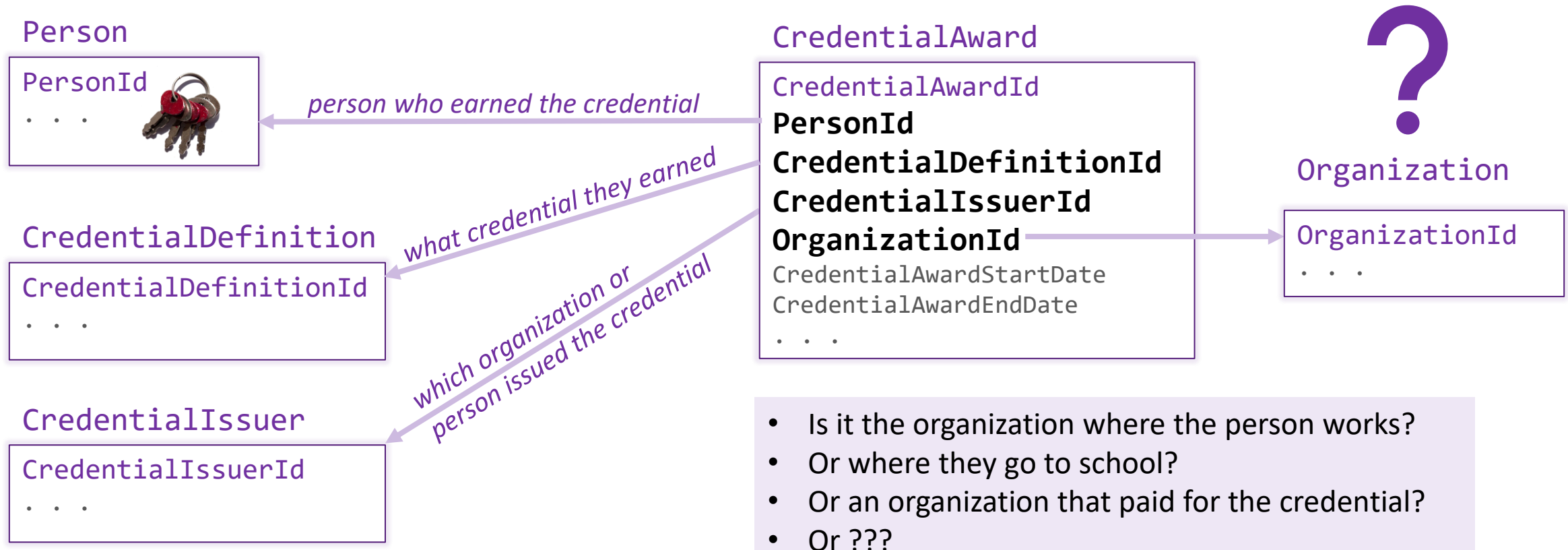
Let's look back at relationships in the IDS.

In the IDS, keys are used to link tables of data together. Sometimes the relationship between the tables is explicitly defined. For example, the OrganizationLocation table acts as a bridge to link an Organization and a Location. That table's link to RefOrganizationLocationType then explicitly **defines the relationship** between the two – whether the Location is the Organization's *mailing* address, *physical* address, or *shipping* address.

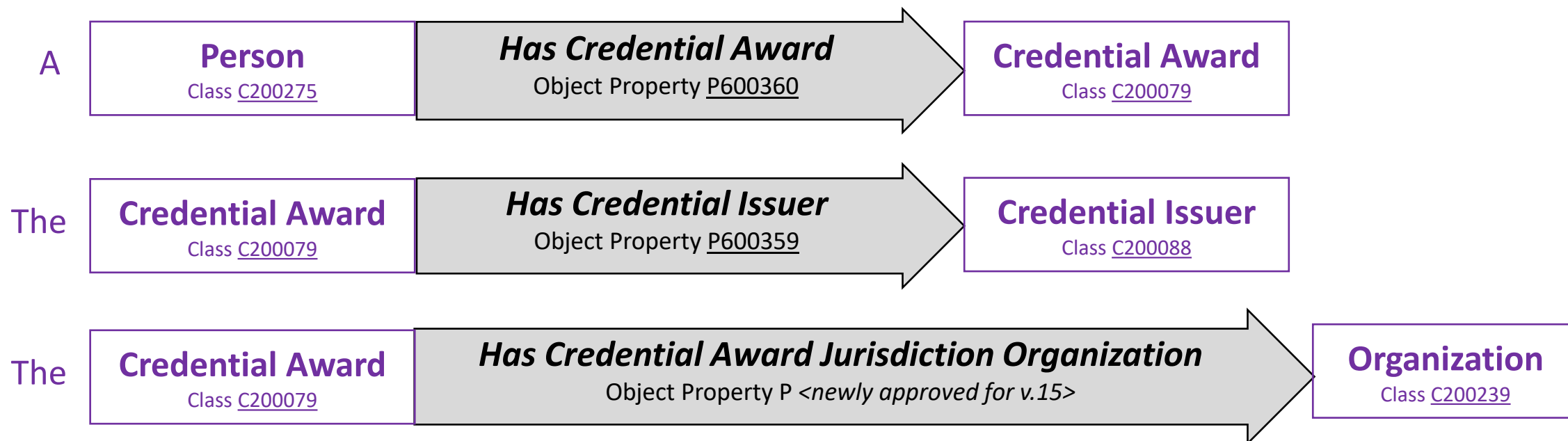


What about when the relationship isn't easy to infer?

In the CredentialAward diagram below, some relationships are easy to understand – such as PersonId pointing to the person who earned a credential and CredentialIssuerId pointing to the organization that issued the credential. **But why is there an additional link to the Organization table?**



An ontology puts those relationships into words



Now the relationship between a Credential Award and an Organization has additional context. Because the linkage was made using "Has Credential Award Jurisdiction Organization," we can look up the definition of this relationship in the CEDS Ontology and learn that "This credential award is valid for use only in this organization and any organization within its jurisdiction." In this particular case, the link to 'Organization' indicates this is a restriction on the awarded credential. For example, this might be a substitute teacher permit that is only valid at one specific school district.

"Three is a magic number..."

If you noticed that these relationships seem to work in threes, you're right. The CEDS Ontology uses the W3C's Resource Description Framework (RDF) as its structure. In RDF, all data can be represented using 'triples' – which are like very simple sentences. Some simplified examples of triples:

The subject node
(an entity being described)

The predicate
(a relationship, a.k.a. property)

The object
(the literal value or another entity node)

The ball

is the color

green.

The ball

is made of

rubber.

The ball

has the shape

sphere.

The sphere

has a type of symmetry:

rotational symmetry.

The sphere

has a type of symmetry:

reflectional symmetry.



Triples can be used to make knowledge graphs.

The subject node
(an entity being described)

The ball

The sphere

The sphere

The predicate
(a relationship, a.k.a. property)

has the shape

has a type of symmetry:

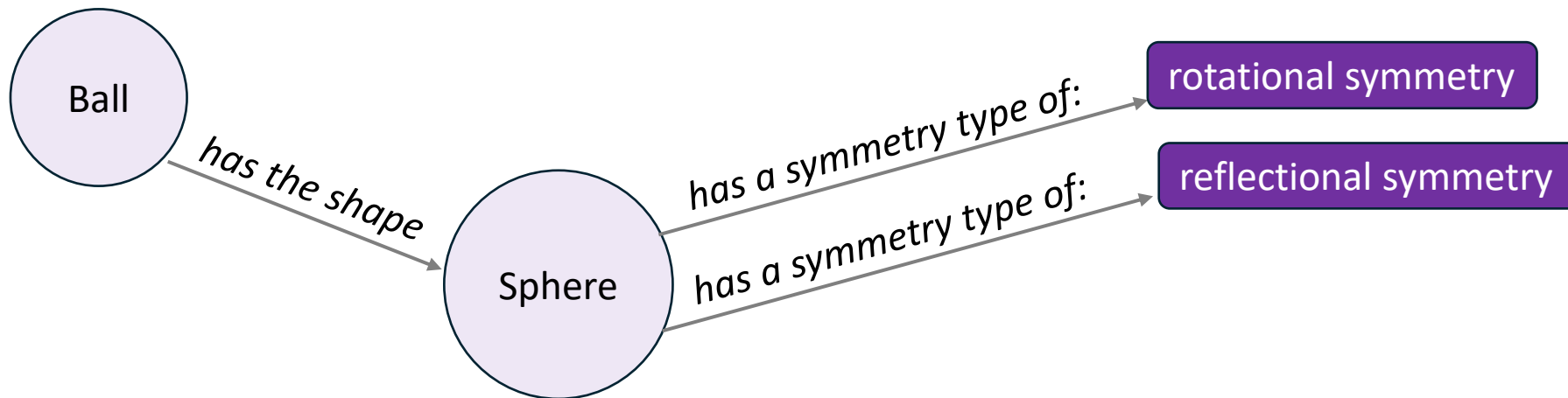
has a type of symmetry:

The object
(the literal value or another entity node)

sphere.

rotational symmetry.

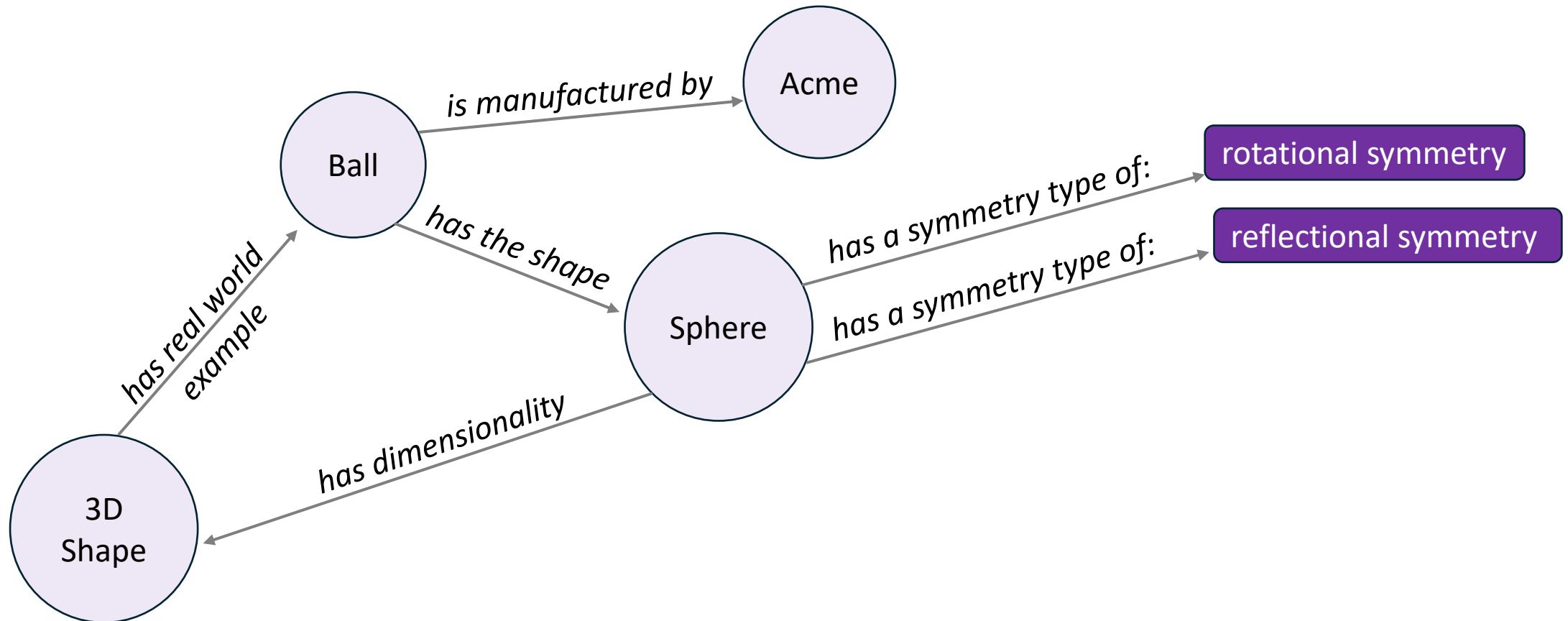
reflectional symmetry.



A knowledge graph enables machine learning (or AI) to make logical inferences based on the relationships. For example, this graph shows that a ball would have both rotational and reflectional symmetry.

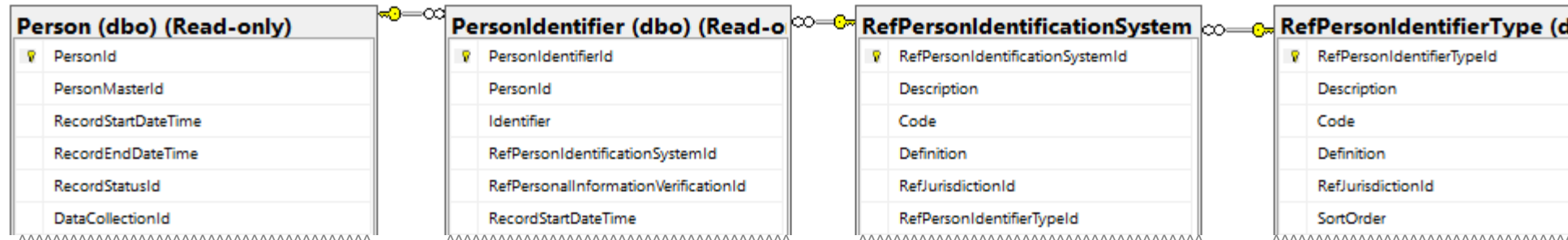
Why use knowledge graphs to structure our data?

Representing data as a graph is a more flexible structure than traditional relational database schemas. For example, when data are stored using an RDF Schema, if we need to add new attributes or connect different datasets, we can do that without having to rewrite the entire database schema.

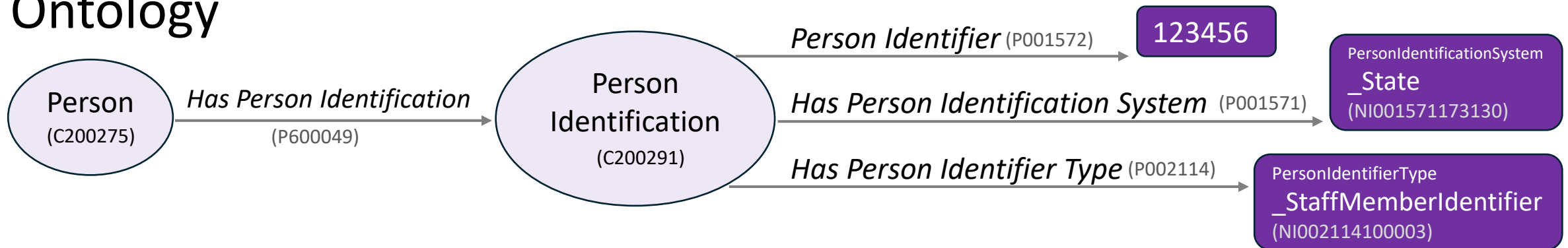


How is the CEDS Ontology similar to the IDS?

IDS



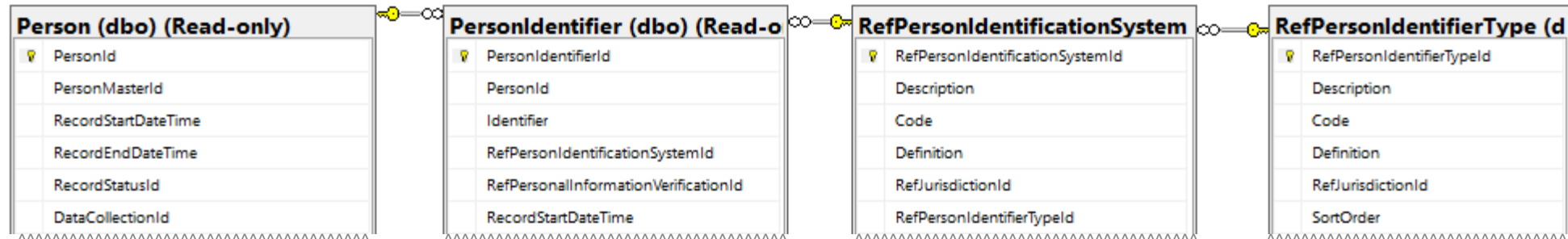
Ontology



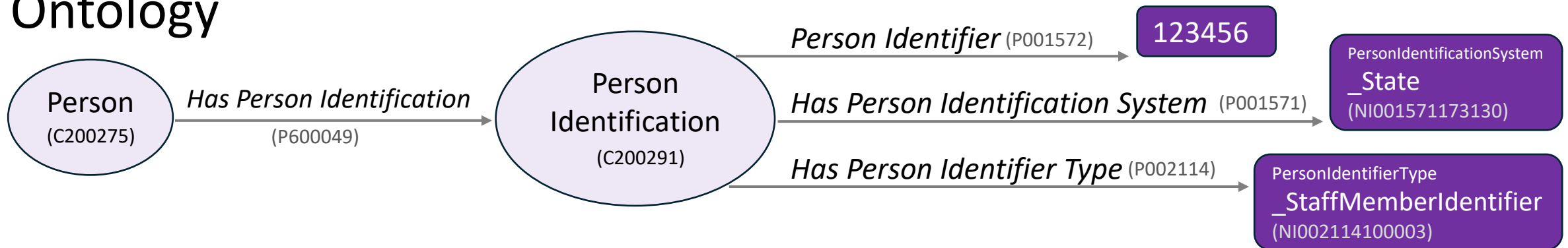
Because the CEDS Ontology was developed from the IDS, much of the normalized structure is retained but just put into graph form. Additionally, the naming of tables and fields in the IDS was adapted to become the naming of different parts of the Ontology (as you can see in the diagram above).

How does the CEDS Ontology differ from the IDS?

IDS



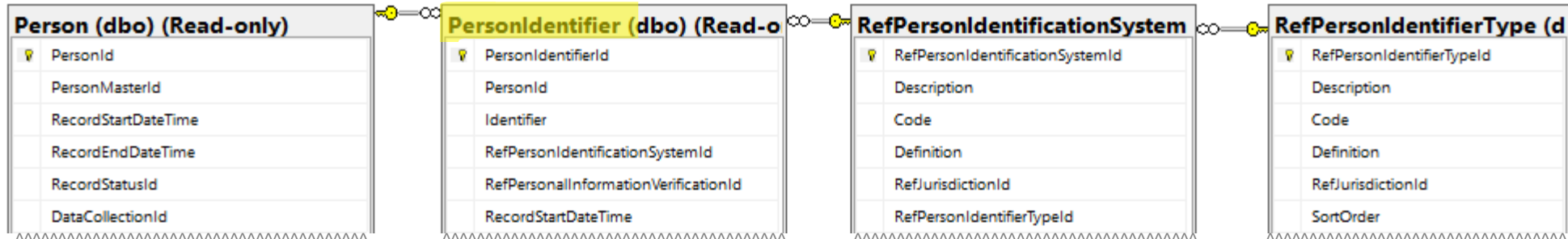
Ontology



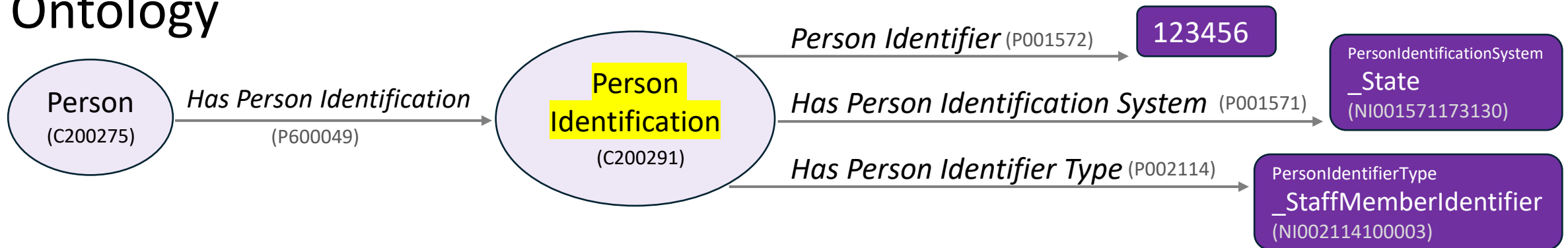
Even though the CEDS Ontology is closely based on the CEDS IDS, you've probably already noticed some differences. Vocabulary is one of those differences. Since we're no longer using rigid tables with rows and columns, we need different terms to refer to how we are grouping and structuring the data.

'Classes' in the ontology are like 'tables' in the IDS

IDS



Ontology



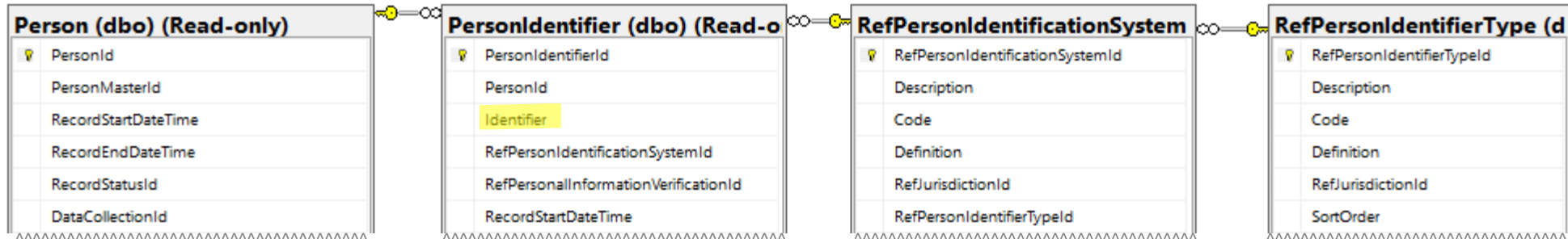
Classes work like tables in that they store data (or store links to data) related to a specific entity or concept.

In the IDS, the PersonIdentifier **table** stores the actual **Identifier** and a link to Ref**PersonIdentificationSystem**.

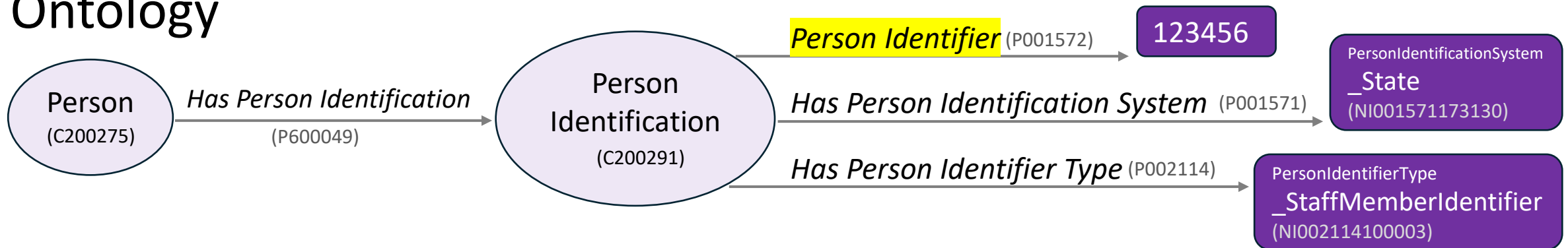
Similarly, the PersonIdentification **class** stores the actual **PersonIdentifier** and the links has**PersonIdentificationSystem** and has**PersonIdentifierType**.

'Properties' are like 'fields' in a table

IDS



Ontology

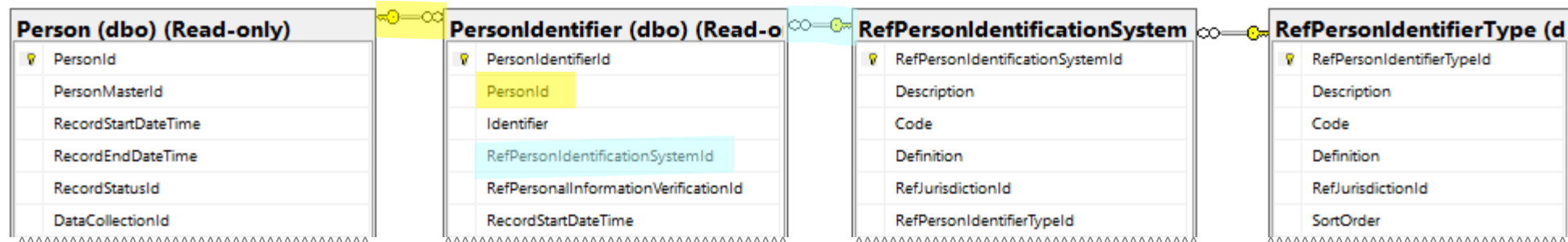


Some fields hold actual data – for example, the **Identifier** field in the PersonIdentifier table would hold a person's identifier and RecordStartDateTime would hold the date and time when that record was created.

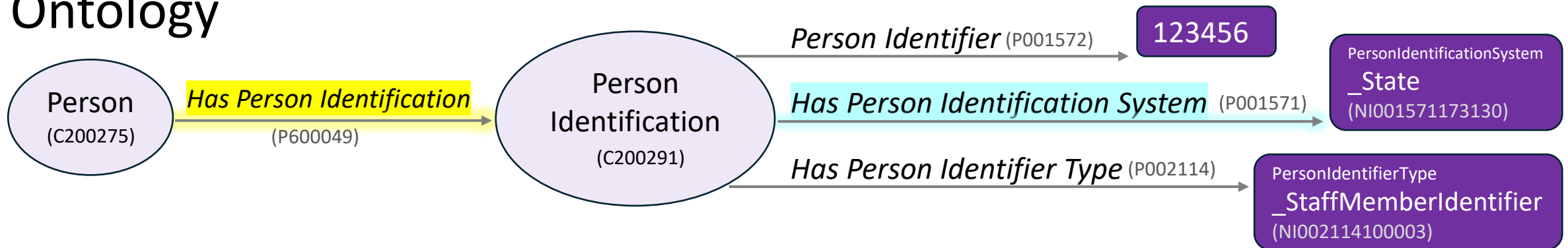
In the ontology, we use **data properties** – like **PersonIdentifier** – to hold actual data. Data properties are assigned a specific datatype such as integer, decimal, string, date, datetime, token, etc.

'Object Properties' are like 'foreign key fields' in a table

IDS



Ontology



Alternatively, a field can hold a foreign key pointing to another table (like **PersonId** points to Person) or a key pointing specifically to a look-up table (like **RefPersonIdentificationSystemId** points to RefPersonIdentificationSystem).

In the ontology, we use **object properties** to hold foreign keys. The foreign key can point to another class (like '**Has Person Identification**') or to class that is operating as a look-up table (like '**Has Person Identification System**')

The Object Property Advantage

Wait... object properties sound like those relationships we were talking about earlier...



An ontology puts those relationships into words

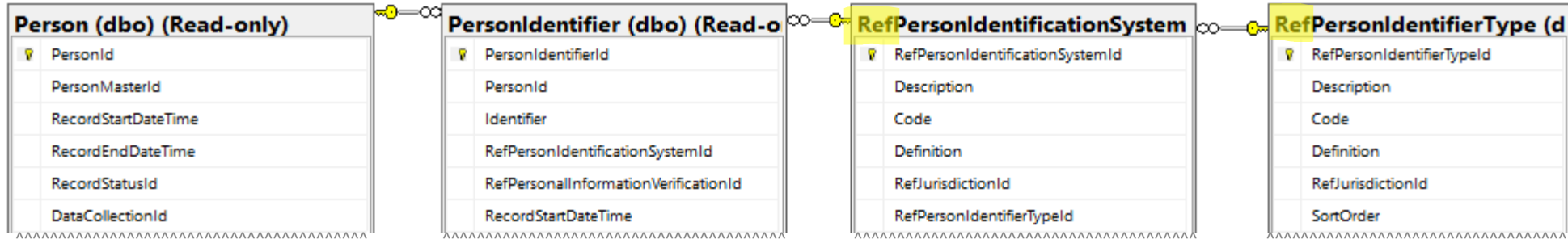


When we learned about how an ontology puts relationships into words, we were indeed talking about **object properties**. The use of object properties is one of the advantages of using the CEDS ontology because the object property names and definitions provide additional context (and clarity) when we link different pieces of data.

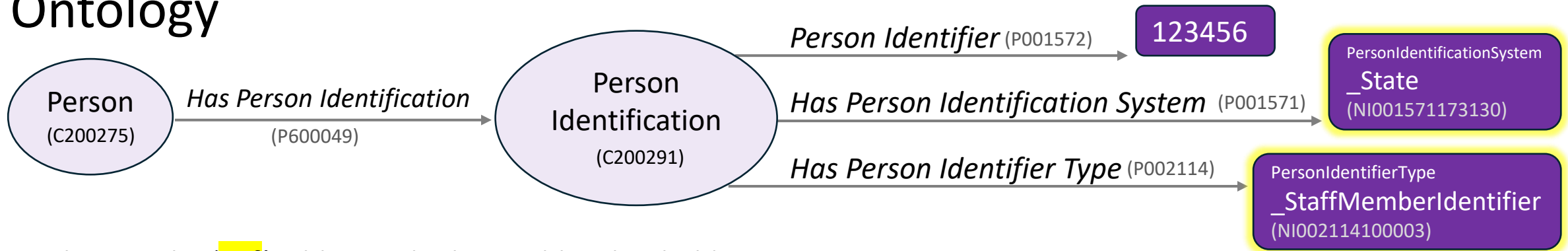
Notice that the names of object properties always start with a verb. Most object properties in CEDS that were derived from the IDS currently start with 'Has' but as the Ontology continues to evolve and mature, we'll see a wider variety of verbs being used to describe relationships. For example, the award of a credential "Has [a] Credential Issuer" but a person or organization "Is [a] Credential Issuer".

Look-up tables are 'Concept Schemes'

IDS



Ontology



In the IDS, the 'Ref' tables are look-up tables that hold an option set.

In the Ontology, 'concept schemes' are a special kind of class used to hold an option set.

If this sounds strange, remember one definition of "scheme" is "a systematic or organized configuration."

A concept scheme is just an organized list of options for one concept. For example, a concept like "language" would have options like English, Spanish, or Arabic in its "scheme".

Concept schemes hold option sets just like the IDS

But there are some differences in how the machine-readable identifiers for each option work!



RefPersonIdentificationSystemId	Description	Code
1	Canadian Social Insurance Number	CanadianSIN
2	District-assigned number	District
3	Family unit number	Family
4	Identification number	Federal
5	Member	NationalMigran
6	Description	School
7	Child Identification System	SSN
8	Staff Member Identifier	State

In the IDS, each option in a Ref table is assigned a **system key**. Each Ref table starts numbering their keys at '1' and increments up. The keys are unique **within the Ref table only**.

In the Ontology, each option is assigned a 14-character identifier called a **Named Individual Code** (or NI Code) that is **unique within CEDS**. Just like a URL is unique across the entire internet and will take you to one specific webpage, the Named Individual Code is unique across the ontology and will take you to one specific option in one specific concept scheme.

ConceptSchemeName	ConceptSchemeCode	NamedIndividualCode	OptionCode
PersonIdentificationSystem	C001571	NI001571173116	CanadianSIN
PersonIdentificationSystem	C001571	NI001571173117	District
PersonIdentificationSystem	C001571	NI001571173133	Family
PersonIdentifierType	C002114	NI002114100000	PersonIdentifier
PersonIdentifierType	C002114	NI002114100001	ChildIdentifier
PersonIdentifierType	C002114	NI002114100003	StaffMemberIdentifier

Everything else about option sets is the same

All options in Concept Schemes will have Option Codes, Descriptions and Definitions just like in the IDS.

Description

- A brief human-readable label or name for the option.
- It uses standard capitalization and spacing.
- It's what you might see displayed in a drop-down menu or on a report.

Definition

- The formal explanation of what the option means in the context of the concept scheme (option set).
- A good definition states the precise meaning or criteria for that option to remove ambiguity / increase clarity.

Option Code	Option Description	Option Definition
3TO5NOTK	3 through 5 (not in kindergarten)	The age range of special education students taught is 3 through 5 (not in kindergarten)
AGE5KTO21	5 (in kindergarten) through 21	The age range of special education students taught is 5 (in kindergarten) through 21

Option Code

- An alphanumeric string without spaces or special characters.
- What one could easily use for queries in SQL, SAS etc.
- Sometimes the Code is the Description but without spaces
- Each Code must be unique within its option set
- **NOT the same as the Named Individual Code**

Concept Schemes and Properties are not always 1:1

For *most* concept schemes, there is only one object property that points to it, and the names of the object property and concept scheme will make the relationship obvious:

Object property	points to	Concept Scheme (a.k.a. option set)
Has Organization Type (P001156)	→	Organization Type (C001156)
Has Special Education Age Group Taught (P000564)	→	Special Education Age Group Taught (C000564)
Has Race And Ethnicity (P001943)	→	Race And Ethnicity (C001943)

The two biggest exceptions are the concept schemes for **Grade Level** (C002175) and **ISO 6392 Language Code** (C000317). Remember object properties describe the relationship between two linked tables, but there are *many* reasons to link to a grade or a language and thus *many* object properties linking to those option sets.

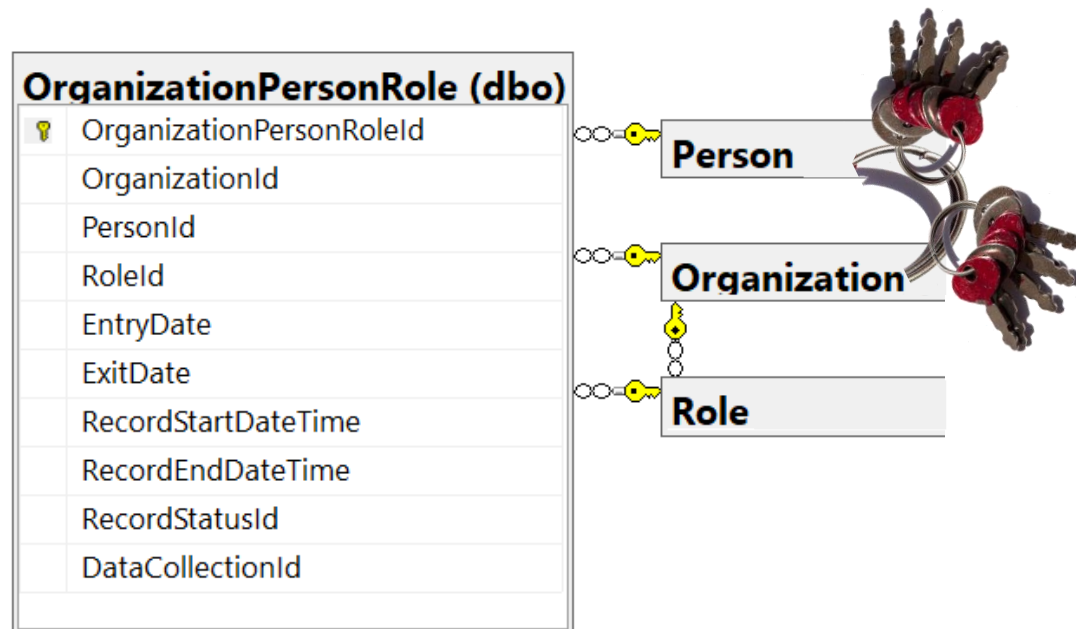
For example, there is one property for the grade level a person was when they **entered** a program/school versus when they **exited**. One property links to the grade levels a school is **approved** to have while another links to the grades the school actually **offers**.

Object properties that link to Grade Level (C002175)

Property_Name	Property_Code
hasAssessmentLevelForWhichDesigned	P000177
hasAssessmentRegistrationGradeLevelToBeAssessed	P001057
hasCourseApplicableEducationLevel	P001301
hasEntryGradeLevel	P000100
hasExitGradeLevel	P001210
hasGradeLevel	P600206
hasGradeLevelsApproved	P001926
hasGradeLevelWhenAssessed	P000126
hasGradeLevelWhenCourseTaken	P000125
HasGradesOffered	P000131
hasHighGradeLevel	P002083
hasLowGradeLevel	P002082

FAQ: Where did IDS table OrganizationPersonRole go in the Ontology?

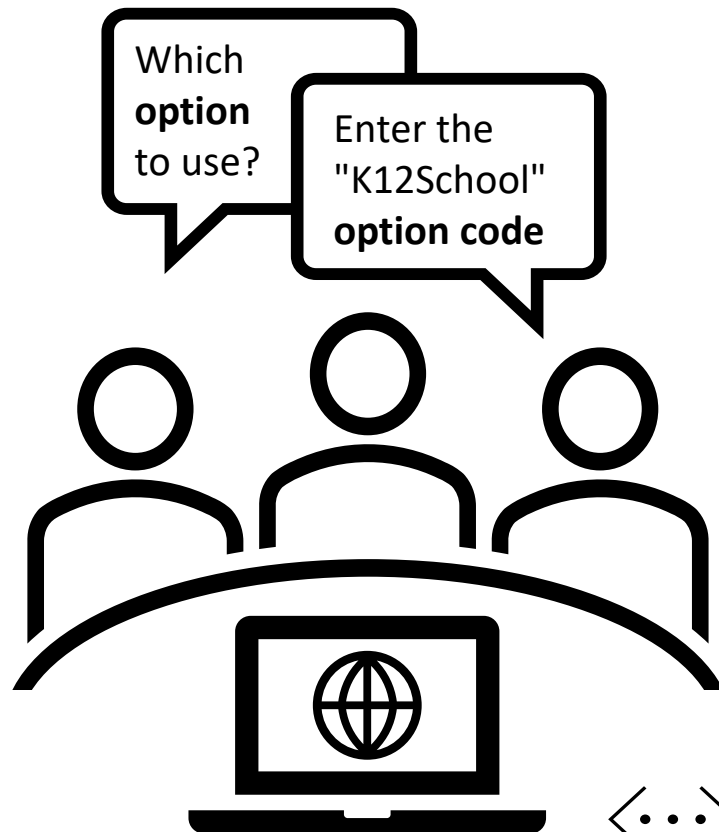
In the IDS, **OrganizationPersonRole** is where we would link a person to a *specific* organization during a *specific* range of time for a *specific* role (like student, staff member, etc.)



In the Ontology, the **Membership class (c200257)** fulfills a similar role by pulling together properties that replicate the functionality of OrganizationPersonRole:

- [Has Organization \(P600062\)](#)
- [Has Person \(P600510\)](#)
- [Has Role \(P600511\)](#)
- [Membership Entry Date \(P600559\)](#)
- [Membership Exit Date \(P600560\)](#)
- [Record Start Date Time \(P001917\)](#)
- [Record End Date Time \(P001918\)](#)
- [Has Record Status \(P201001\)](#)
- [Has Data Collection \(P201003\)](#)

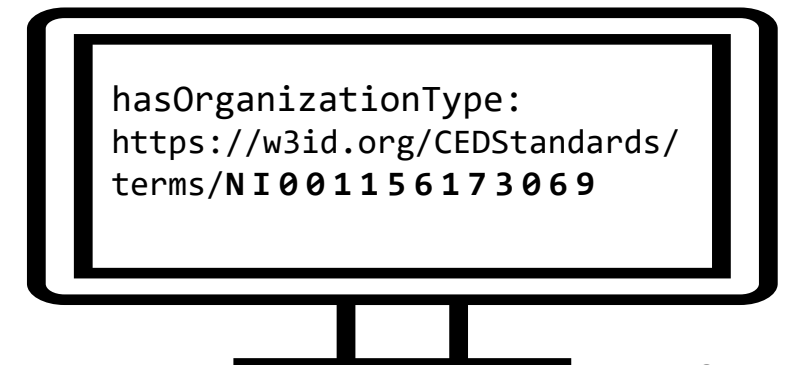
Public-facing language vs. Machine-readable language



When talking with stakeholders, or when publishing an ontology to the public, we will continue to use the common terms like option set, option code, etc. The CEDS community wants the standard to be accessible and easy-to-use which is why the ontology has continued to include the human-friendly 'Code' and 'Description' in addition to the machine-readable Named Individual Code.

The technical vocabulary of ontologies is essential for those working with systems on the backend, but it should be used judiciously lest it become an obstacle for stakeholders.

<...> <...> <...> <...> <...> <...>



Optional Activity #1

1. Explore the relationships between classes, properties, and concept schemes via the CEDS Ontology website.

<https://ceds.ed.gov/desHome.aspx#/>

Browse for a property or concept scheme relevant to your work and then scroll down to the "Found IN" section to find out which classes use that property or which properties link to the concept scheme.

[All](#) > [Classes](#) > [O](#) > [C200239](#) > P600567 v14.0.0.0

Property

Has Organization Identifier Type

A code that identifies the type of identifier assigned to an organization. This element distinguishes among various identification systems or authorities (e.g., state-assigned ID, NCES ID, federal EIN) that issue unique identifiers to education organizations such as schools, districts, or agencies.



Found In

You'll find this property referenced by the following classes.

[Organization](#)
[Organization Identification](#)

Common Education Data Standards

User Name

[Create an Account](#)

Home About The Standard Data Model Tools Resources

CEDS Ontology

Explore the Standard through the CEDS Ontology, a fully searchable resource that brings together the key terms, definitions, option sets, relationships, technical specifications, and more.

Search...

Filter by
 All

Quick Links

Classes Properties

[Organization](#) + Updated

[Organization Accreditation](#) +

[Organization Calendar](#) +

[Organization Calendar Crisis](#) +

Or select a starting class and then click the + symbols to explore the chain of properties and classes that can flow out from it.

Classes Properties

[Organization](#) + Updated

[Organization Accreditation](#) -

[Has Accreditation Agency](#) [object property: Accreditation Agency] +

[Accreditation Agency Name](#) [data property: string]

[Accreditation Award Date](#) [data property: date]

Optional Activities #2 & #3

2. If working with an ontology is new to you, this could feel like a lot of new vocabulary.
Remember how teachers would sometimes use little games and quizzes to practice vocabulary in school?
If that's appealing to you, take 5-10 minutes to do a light-hearted [Wayground interactive quiz](#) over the vocabulary we just covered.
NOTE: It will prompt you for your name, but feel free to enter a fictitious or fun name as you wish.
3. If you'd like to explore how object properties work to link classes to other classes (or to concept schemes), try completing the "Classes and Object Properties - PowerPoint Activity.pptx" that goes along with this training.

Time to Reflect & Digest on the Ontology



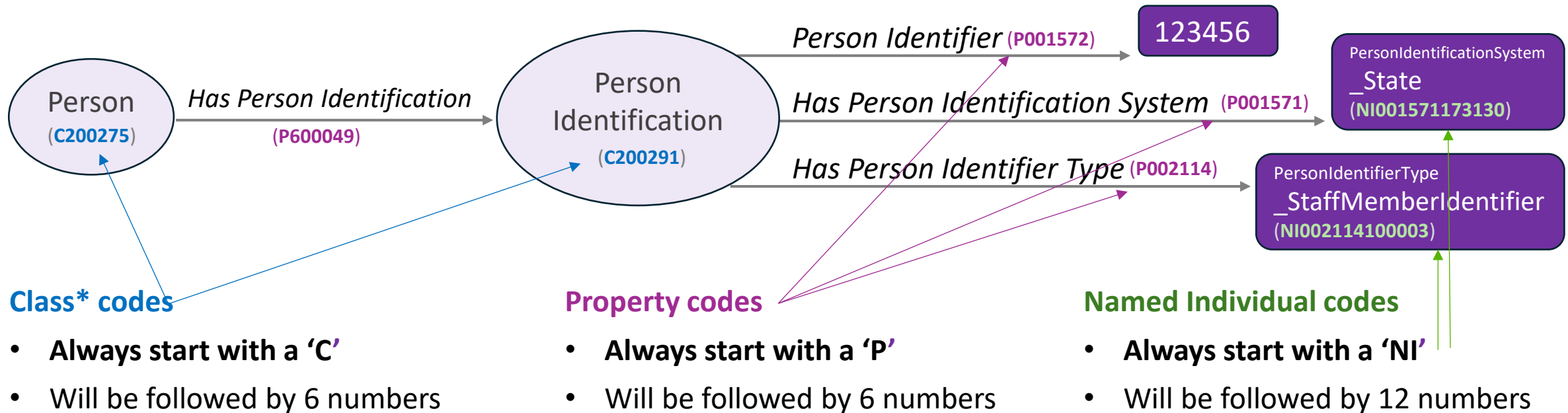
- What are some of the benefits of using the CEDS Ontology? Slide 28
 - What is the benefit to representing data as a knowledge graph? Slides 34-36
 - What is the benefit to using object properties to describe relationships? Slides 31-33
- How is the Ontology similar to and different from the IDS? Slides 37-38
 - What are classes and concept schemes? Slides 39, 43-44
 - What are data properties and object properties? Slides 40-42
 - What is the relationship between object properties and concept schemes? Slide 46
 - How do option sets in the Ontology compare to the IDS Slide 43-45
- REFLECT:
 - How might these structures and conventions impact your work?
 - What questions do you have?

NEXT UP: **Codes & Namespaces in the CEDS Ontology!**

We're going to get into more technical aspects of the ontology. These are most applicable to analysts helping to develop extension ontologies, SHACL or JSON-LD templates. If you will only work with reporting data in the semantic layer, feel free to skip to slide 66

Codes, codes, codes everywhere

You may have noticed alphanumeric codes popping up throughout this training. Each class, concept scheme, property, and option have a unique identifier in the CEDS Ontology.



* **Concept Schemes** also follow these rules since they are a special kind of class.

Sometimes there are hidden patterns in the codes

When an object property points to a concept scheme, their identifiers are *usually* the same except for the leading character.

Object property (a.k.a. foreign key)	points to	Concept Scheme (a.k.a. option set)
Has Organization Type (P001156)	→	Organization Type (C001156)
Has Special Education Age Group Taught (P000564)	→	Special Education Age Group Taught (C000564)
Has Race And Ethnicity (P001943)	→	Race And Ethnicity (C001943)

The many object properties that point to Grade Level (c002175) or ISO 6392 Language Code (c000317) are obviously exceptions to this rule. Additionally, there are a small number of concept schemes that were created at a different time than their corresponding object property and so the codes are out of sync.

Another pattern is that each Named Individual Code has embedded within it the numeric portion of the code for the corresponding Concept Scheme.

ConceptSchemeName	ConceptSchemeCode	NamedIndividualCode	OptionCode
PersonIdentificationSystem	C001571	NI001571173116	CanadianSIN
PersonIdentificationSystem	C001571	NI001571173117	District
PersonIdentificationSystem	C001571	NI001571173133	Family
PersonIdentifierType	C002114	NI002114100000	PersonIdentifier
PersonIdentifierType	C002114	NI002114100001	ChildIdentifier
PersonIdentifierType	C002114	NI002114100003	StaffMemberIdentifier

What's a "Namespace"?

A common analogy is to compare a namespace to a family's last name. At home, a child might be called by just his first name – like Deshaun. However, at school there might be multiple students named 'Deshaun' and so last names are used to differentiate them. Deshaun Grey is different than Deshaun Green, for example.

In this training, when we've talked about the CEDS codes for classes or properties, we've been using the equivalent of their 'first name' – like P001156 for the 'Has Organization Type' property or C001943 for the 'Race and Ethnicity' concept scheme. These identifiers are like the 'first names' because they provide an easy shorthand way to refer to each property or class when we're talking to others inside the CEDS family.

Adding a namespace to these identifiers provides "a context for identifiers, a logical grouping of names..." – essentially saying that all these identifiers belong to a specific family or domain so that they don't get confused with other, similar identifiers in other domains.



CEDS Namespace + CEDS Code = UNIQUE identifier

	Example	Full identifier
<i>Class</i>	Organization Identification	https://w3id.org/CEDStandards/terms/C200252
<i>Data Property</i>	Organization Identifier	https://w3id.org/CEDStandards/terms/P000826
<i>Object Property</i>	Has Organization Identification System	https://w3id.org/CEDStandards/terms/P000827
<i>Concept Scheme</i>	Organization Identification System	https://w3id.org/CEDStandards/terms/C000827
<i>Named Individual</i>	SEA	https://w3id.org/CEDStandards/terms/NI000827151971

The CEDS namespace is <https://w3id.org/CEDStandards/terms/>

When the CEDS code for a class, property, or named individual, is appended to the end of the CEDS namespace, it becomes an identifier that is **globally unique** – an identifier that is literally unique across all domains and standards across the entire world.

This is very important when we're talking about making data accessible for machine learning or AI. Having a namespace that is reserved for just CEDS terms helps prevent 'collisions' where terms (or objects) in two different systems have the same identifier and so get confused with one another.

The power of namespaces



Namespaces also help improve interoperability of systems because of how they create a shared, predictable way to refer to things. Different systems can interact with one another, knowing that what System A is calling a 'K12 School' is the same as what System B is calling a 'K12 School' because both are pointing to the same CEDS definition at <https://w3id.org/CEDStandards/terms/C200199>

This also makes it possible for newer ontologies and systems to build on top of the work done by others before them. When terms have already been defined in one public namespace, other systems can draw on them to help build out their definitions and relationships.

For example, the CEDS ontology uses the CEDS namespace to provide unique identifiers to each CEDS term. But the CEDS ontology also uses the XML namespace to define the datatypes for data properties.

Data Property	Datatype
Record End Date Time	http://www.w3.org/2001/XMLSchema#dateTime
First Name	http://www.w3.org/2001/XMLSchema#string
Full Time Equivalency	http://www.w3.org/2001/XMLSchema#decimal
Group Size Range Minimum	http://www.w3.org/2001/XMLSchema#nonNegativeInteger

CEDS draws on multiple namespaces



Name	Namespace
Dublin Core	http://purl.org/dc/elements/1.1/
Web Ontology Language	http://www.w3.org/2002/07/owl#
Resource Description Framework	http://www.w3.org/1999/02/22-rdf-syntax-ns#
Resource Description Framework Schema	http://www.w3.org/2000/01/rdf-schema#
Simple Knowledge Organization System	http://www.w3.org/2004/02/skos/core#
Schema	https://schema.org/
XML Schema	http://www.w3.org/2001/XMLSchema#



Web Ontology Language
(OWL)
by [World Wide Web Consortium \(W3C\)](http://www.w3.org/)

Instead of trying to duplicate the work done by other namespaces, the CEDS ontology draws on these commonly used public namespaces to build out the definitions of each CEDS term. We've already talked about how the Resource Description Framework can be used to build knowledge graphs from CEDS terms and the XML Schema is used to define datatypes. Dublin Core has standardized metadata terms like the "Title" of a resource or the "Creator." The Web Ontology Language provides terms that allow CEDS to document version changes in the ontology and add annotations like the minimum or maximum count. And so on.

Aliases help keep things human-readable

Alias	Name	Namespace
dc	Dublin Core	http://purl.org/dc/elements/1.1/
owl	Web Ontology Language	http://www.w3.org/2002/07/owl#
rdf	Resource Description Framework	http://www.w3.org/1999/02/22-rdf-syntax-ns#
rdfs	Resource Description Framework Schema	http://www.w3.org/2000/01/rdf-schema#
skos	Simple Knowledge Organization System	http://www.w3.org/2004/02/skos/core#
schema	Schema	https://schema.org/
xsd	XML Schema	http://www.w3.org/2001/XMLSchema#

Remember one of the primary goals of the CEDS ontology is to create a **machine-readable** document. At the same time, humans also need to be able to read the Ontology. To balance these competing needs, the CEDS ontology defines shorter **aliases** for namespaces at the beginning of the document. CEDS is essentially telling any machines reading this document to treat "dc:" like <http://purl.org/dc/elements/1.1/>, and so on.

This means instead of writing out `<http://purl.org/dc/elements/1.1/creator>` every time the creator of a term needs to be identified, the CEDS ontology can just use `<dc:creator >`

What does an CEDS ontology entry look like?



We've talked a lot about the namespaces and machine readability – let's walk through one example of how that works in the CEDS Ontology using the **"Has Person Identifier Type" object property**.

```
<rdf:Property rdf:about="https://w3id.org/CEDStandards/terms/P002114">
  <dc:creator>Common Education Data Standards</dc:creator>
  <dc:identifier rdf:datatype="http://www.w3.org/2001/XMLSchema#token">P002114</dc:identifier>
  <rdfs:label>Has Person Identifier Type</rdfs:label>
  <schema:domainIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C200291" />
  <schema:rangeIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C002114" />
  <skos:notation>hasPersonIdentifierType</skos:notation>
  <dc:description>A coding scheme of identifiers assigned to an individual, distinguishing between various
    roles and contexts within an educational or organizational setting. It ensures accurate
    identification and classification of individuals based on their specific type of
    identifier.</dc:description>
  <skos:definition>A coding scheme of identifiers assigned to an individual, distinguishing between various
    roles and contexts within an educational or organizational setting. It ensures accurate
    identification and classification of individuals based on their specific type of
    identifier.</skos:definition>
</rdf:Property>
```

What is this?

```
<rdf:Property rdf:about="https://w3id.org/CEDStandards/terms/P002114">
```

The **opening and closing tags** are telling machines that everything enclosed within the tags is about one Property – and by ‘Property’ we mean what RDF defines as a ‘Property.’

And then **rdf:about** gives us the globally unique CEDS identifier to make it clear which CEDS property we’re talking about in this entry.

```
</rdf:Property>
```

Populate the metadata

```
<rdf:Property rdf:about="https://w3id.org/CEDStandards/terms/P002114">  
  <dc:creator>Common Education Data Standards</dc:creator>  
  <dc:identifier rdf:datatype="http://www.w3.org/2001/XMLSchema#token">P002114</dc:identifier>
```

We use Dublin Core (dc) to say that CEDS is the **creator** of this property, and we say that within CEDS this property has an **identifier** of **P002114**.

Additionally, nested inside of that metadata is a note that the **datatype** of the **P002114** identifier is 'token'. The token datatype is based on the string datatype but "the XML processor will remove line feeds, carriage returns, tabs, leading and trailing spaces, and multiple spaces."

```
</rdf:Property>
```

Add human-readable and technical names



```
<rdf:Property rdf:about="https://w3id.org/CEDStandards/terms/P002114">  
  <dc:creator>Common Education Data Standards</dc:creator>  
  <dc:identifier rdf:datatype="http://www.w3.org/2001/XMLSchema#token">P002114</dc:identifier>  
  <rdf:label>Has Person Identifier Type</rdf:label>
```

```
<skos:notation>hasPersonIdentifierType</skos:notation>
```

The **rdf:label** provides a friendly human-readable name with spaces between words and Proper Case used for the capitalization. This is what is displayed on the CEDS website or in business materials.

The **skos:notation** provides a technology-ready name using camelCase and NO spaces. CEDS refers to this as the 'Technical Name' of the property.

```
</rdf:Property>
```

Identify the domain and range

```
<rdf:Property rdf:about="https://w3id.org/CEDStandards/terms/P002114">  
  <dc:creator>Common Education Data Standards</dc:creator>  
  <dc:identifier rdf:datatype="http://www.w3.org/2001/XMLSchema#token">P002114</dc:identifier>  
  <rdfs:label>Has Person Identifier Type</rdfs:label>  
  <schema:domainIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C200291" />  
  <schema:rangeIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C002114" />
```

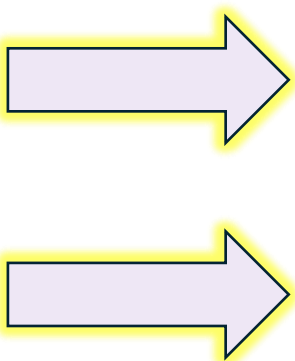


Remember that object properties link two classes (or a class and a concept scheme) together. The **schema:domainIncludes** gives the globally unique identifier for the starting class. The **schema:rangeIncludes** gives the globally unique identifier for the destination class – which in this case, is a concept scheme.

```
</rdf:Property>
```

Provide an explicit definition

```
<rdf:Property rdf:about="https://w3id.org/CEDStandards/terms/P002114">
  <dc:creator>Common Education Data Standards</dc:creator>
  <dc:identifier rdf:datatype="http://www.w3.org/2001/XMLSchema#token">P002114</dc:identifier>
  <rdfs:label>Has Person Identifier Type</rdfs:label>
  <schema:domainIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C200291" />
  <schema:rangeIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C002114" />
  <skos:notation>hasPersonIdentifierType</skos:notation>
  <dc:description>A coding scheme of identifiers assigned to an individual, distinguishing between various
    roles and contexts within an educational or organizational setting. It ensures accurate
    identification and classification of individuals based on their specific type of
    identifier.</dc:description>
  <skos:definition>A coding scheme of identifiers assigned to an individual, distinguishing between various
    roles and contexts within an educational or organizational setting. It ensures accurate
    identification and classification of individuals based on their specific type of
    identifier.</skos:definition>
</rdf:Property>
```

Two yellow arrows with black outlines, pointing horizontally to the right. The first arrow is positioned above the second one, and both are located to the left of the text content.

Why did the definition appear twice?

```
<dc:description>A coding  
roles an  
identific  
identifie  
<skos:definition>A codin  
roles an  
identific  
identifie
```

The Dublin Core "description" and the SKOS "definition" exist for *similar* purposes but are *not* identical in usage.

A "definition" in the Simple Knowledge Organization System is expected to be a "complete explanation of the intended meaning of a **concept**." This means machine learning (or AI) systems expect "skos:definition" to be populated with a clear, unambiguous formal definition of something that is a **concept**.

In contrast, the Dublin Core "description" is populated with "an account of the **resource**" – meaning it can be used for any resource, including documents, datasets, media, etc. in addition to concepts. Additionally, the dc:description can include extra examples or background context to make it useful to the humans reading it.

In CEDS, the skos:definition and dc:description are generally populated by the same text, but that is not always true in other ontologies. CEDS provides both so that any systems accessing the CEDS ontology that are looking specifically for a skos:definition (*I want the authoritative definition!*) or a dc:description (*I want the full description of this thing!*) can find exactly what they are looking for – thereby making the CEDS Ontology interoperable with more systems.

What about entries for data properties, classes, etc.?

How does it compare to an object property entry?

Data Property

Data properties are defined the same as object properties except that the destination for `<schema:rangeIncludes>` is the XML datatype for that data property. (See slide 51 for a table of sample XML datatypes.)

Class

A class that is not a concept scheme is enclosed in `<rdfs:Class>` tags instead of property tags. Classes can stand alone so they won't have a domain and range. However, most classes will be a `<rdfs:subClassOf>` another class, and they'll inherit all the properties in the parent class. Most commonly, classes are subclasses of the CEDS Base Class (C000000) so that they can inherit record management properties like Record Start Date Time, Record End Date Time, Has Record Status, etc. But some classes are subclasses of subclasses – such as K12 School being a subclass of Organization or Employment being a subclass of Membership.

Concept Scheme

Concept Schemes are enclosed in `<owl:Class>` tags. Concept Schemes are a special type of class so they'll have everything else that a class has *plus* they'll be identified as a concept scheme by the inclusion of: `<rdf:type rdf:resource = "http://www.w3.org/2004/02/skos/core#ConceptScheme" />`
Also: Concept Schemes are a 'concept' so they get a `skos:prefLabel` populated in addition to their `rdfs:label` – again just for interoperability.

What about options in the Concept Scheme?



How does it compare to an object property entry?

Named Individual

Named Individuals (or options) are enclosed in `<owl:NamedIndividual>` tags.

We'll show that this is an option in concept scheme in RDF by including

```
<rdf:type rdf:resource="http://www.w3.org/2004/02/skos/core#Concept" />
```

Instead of a `<schema:domainIncludes>` tag, we'll specify exactly *which* concept scheme this option belongs to by giving the full identifier for the concept scheme:

```
<rdf:type rdf:resource="https://w3id.org/CEDStandards/terms/C000087" />
```

```
<skos:inScheme rdf:resource="https://w3id.org/CEDStandards/terms/C000087" />
```

(And we did it twice for interoperability)

Additionally, because each option is its own example of a 'concept' within the larger 'concept scheme', Named Individuals will also have `<skos:prefLabel>` that matches their `<rdfs:label>`

Optional Activity #4



1. Open the [CEDS Ontology website](#) and browse to a property, class, or concept scheme related to your work.
2. In another tab, navigate to the [CEDS-Ontology GitHub](#) and download the CEDS-Ontology.
3. Search for the property or class you see on the website **by name** in the CEDS-Ontology document you downloaded.

Compare how identifiers, definitions, domains and ranges present in the document with how they appear on the website. What do you notice?

Ontology Website
v14.0.0.0

[All](#) > [Classes](#) > [O](#) > [C200239](#) > [P600567](#)

Property

Has Organization Identifier Type

A code that identifies the type of identifier assigned to an organization. This element distinguishes among various identification systems or authorities (e.g., state-assigned ID, NCES ID, federal EIN) that issue unique identifiers to education organizations such as schools, districts, or agencies.

Ontology Document

```
<rdf:Property rdf:about="https://w3id.org/CEDStandards/terms/P600567">
  <dc:creator>Common Education Data Standards</dc:creator>
  <dc:identifier rdf:datatype="http://www.w3.org/2001/XMLSchema#token">P600567</dc:identifier>
  <dc:description>A code that identifies the type of identifier assigned to an organization. This
element distinguishes among various identification systems or authorities (e.g., state-assigned ID, NCES ID,
federal EIN) that issue unique identifiers to education organizations such as schools, districts, or
agencies.</dc:description>
  <skos:definition>A code that identifies the type of identifier assigned to an organization. This
element distinguishes among various identification systems or authorities (e.g., state-assigned ID, NCES ID,
federal EIN) that issue unique identifiers to education organizations such as schools, districts, or
agencies.</skos:definition>
  <rdfs:label>Has Organization Identifier Type</rdfs:label>
  <schema:domainIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C200239" />
<schema:domainIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C200252" />
  <schema:rangeIncludes rdf:resource="https://w3id.org/CEDStandards/terms/C200416" />
  <skos:notation>hasOrganizationIdentifierType</skos:notation>
```

Time to Reflect & Digest on Codes & Namespaces



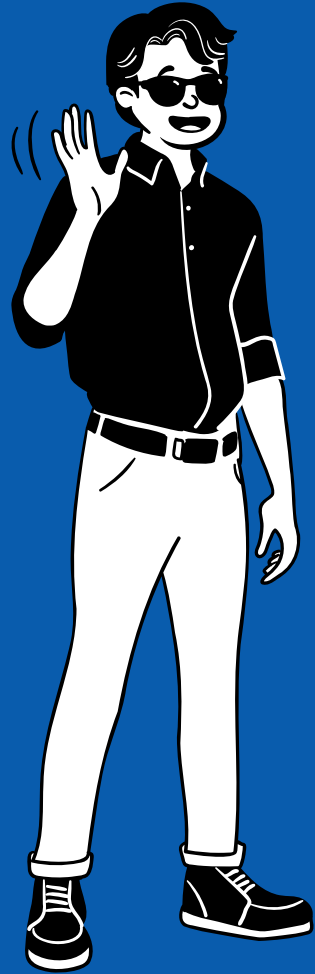
- What are namespaces? Why does the CEDS ontology draw on namespaces? Slide 54-57
 - What are some of the patterns used to create codes for classes and properties? Slides 52-53, 55
 - What other ontologies and namespaces does CEDS draw on? Why? Slide 56-57, 61, 65
 - Why does the CEDS ontology use aliases? Slide 58
- What does a CEDS ontology entry look like? How can a human make sense of it? Slides 59-67
 - Why does it look like some data has been duplicated? Slide 65
 - What is the “domain” and “range?” Slide 63
- REFLECT:
 - How might these structures and conventions impact your work? What questions do you have?

NEXT UP: **Implementing the CEDS Ontology in JSON-LD**

JavaScript Object Notation for Linked Data



JSON-LD logo borrowed from the [JSON-LD.org](https://github.com/json-ld/json-ld.org) repository on GitHub
under the [W3C Software and Document License](https://www.w3.org/2019/02/w3c-software-and-document-license/)



Before we transition into the JSON-LD section, please know that business analysts will rarely work with raw JSON-LD files.

Most of the time, you'll access this data through semantic datasets or staging tables that will transform the data in a more familiar form – like columns and rows in a table.

However, for those who will help with ETLs into the CEDS data warehouse or help with development of transactional systems, understanding how JSON-LD is structured will be helpful to your work.

What is JSON?

JSON (JavaScript Object Notation) is a lightweight, "text-based format for storing and exchanging data in a way that's both human-readable and machine-parsable. As a result, JSON is relatively easy to learn and to troubleshoot."

This flexible format is popular because it "simplifies data interchange across diverse platforms and programming languages" and so "delivers faster data interchange and web service results." It's commonly used across the web and in APIs.

TLDR: JSON is a text document that makes it easy to store and move data.

```
{  
  "FirstName": "Annie",  
  "LastOrSurname": "Sample",  
  "MiddleName": "M",  
  "AddressStreetNumberAndName": "123 Main St",  
  "AddressCity": "Smalltown",  
  "AddressPostalCode": "49001",  
  "hasStateAbbreviation": "StateAbbreviation_MI"  
}
```

In its simplest form, JSON is simply a set of **key-value pairs**.

- The key must be a unique string in quotes.
- The key and value are separated by a colon.
- The value is typically in quotes unless it's numeric.
- Pairs are separated by commas.
- The set is enclosed in curly brackets {}.

How is JSON-LD different from JSON?



JSON-LD builds on regular JSON by adding the 'LD' – or '**Linked Data**'.

A JSON-LD document uses reserved keywords and hyperlinks that work together to enable applications "to start at one piece of Linked Data, and follow embedded links to other pieces of Linked Data that are hosted on different sites across the Web"

Disclaimer: We're starting with some simplified JSON-LD examples before we get to the CEDS-aligned examples.

```
{
  "@id": "https://www.namespace.example/person/12345",
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",

  "FirstName": "Annie",
  "LastOrSurname": "Sample",
  "MiddleName": "M",

  "hasParent": {
    "@id" : "https://www.namespace.example/person/67890",
    "@type" : "Person"
  },

  "hasLocationAddress": {
    "@id" : "https://www.namespace.example/address/ae45xq",
    "@type" : "LocationAddress"
  }
}
```

Reserved Keywords



While plain JSON can store data, "JSON-LD tells you what the data *means* — using a shared vocabulary that any system can resolve.

A JSON-LD document is still a valid JSON document. The difference is a small set of reserved keywords — @context, @id, @type — that turn that document into a graph of typed, globally-identified entities."

```
{
  "@id": "https://www.namespace.example/person/12345",
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",

  "FirstName": "Annie",
  "LastOrSurname": "Sample",
  "MiddleName": "M",
}
```

@id holds the globally unique identifier for this document. It *must* be an Internationalized Resource Identifier (IRI), and that IRI *may* be a working URL but it does not have to be unless your data model requires it.

@type explains what **class** of data will be stored in this JSON-LD document. In the example to the left, we're storing data about a Person.

For machine learning applications to understand the value in the @type field actual means, we need to tell it how to look up the definition of "Person" — and the @context file provide those instructions.

The Power of Linked Data



```
{
  "@id": "https://www.namespace.example/person/12345",
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",
  "FirstName": "Annie",
  "LastOrSurname": "Sample",
  "MiddleName": "M",
  "hasParent": {
    "@id": "https://www.namespace.example/person/67890",
    "@type": "Person"
  },
  "hasLocationAddress": {
    "@id": "https://www.namespace.example/address/ae45xq",
    "@type": "LocationAddress"
  }
}
```

`@id` and `@type` at the beginning of the document are providing information about this particular JSON-LD document.

However, we can also use those reserved keywords later to link to other JSON-LD documents.

For example, 'hasParent' hyperlinks to the Person document holding data about one of Annie's parents.

And 'hasLocationAddress' hyperlinks to an Address document that holds all the data related to a specific address.

Context is King



```
{
  "@id":    "https://www.namespace.example/person/12345"
  "@type":  "Person",
  "@context": "https://www.namespace.example/context.json",
  "FirstName":    "Annie",
  "LastOrSurname": "Sample",
  "MiddleName":   "M",
  "hasParent": {
    "@id" : "https://www.namespace.example/person/67890",
    "@type" : "Person"
  },
  "hasLocationAddress": {
    "@id" : "https://www.namespace.example/address/ae45xq",
    "@type" : "LocationAddress"
  }
}
```

We only need to populate the `@context` once.

This field should point to one file that will provide hyperlinks to the ontologies and namespaces potentially referenced by the JSON-LD document. Machine-learning applications (and people) can then use these links to look up definitions, datatypes, and other details about each class or property used – providing the *context* needed to understand the data stored in this JSON-LD document.

Arrays enable 1-to-many relationships



```
{
  "@id": "https://www.namespace.example/person/12345",
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",
  "FirstName": "Annie",
  "LastOrSurname": "Sample",
  "MiddleName": "M",
  "hasParent": [
    {
      "@id" : "https://www.namespace.example/person/67890",
      "@type" : "Person",
      "hasPersonRelationshipType" : "Father",
    },
    {
      "@id" : "https://www.namespace.example/person/52311",
      "@type" : "Person",
      "hasPersonRelationshipType" : "Mother",
    },
    {
      "@id" : "https://www.namespace.example/person/38942",
      "@type" : "Person",
      "hasPersonRelationshipType" : "Stepfather",
    }
  ]
}
```

Suppose we wanted to store links to more than one parent for this student.

JSON allows for a 1-key-to-many-values relationship through arrays.

Square brackets `[]` are used to enclose the array of values and commas are used to separate each of the element in the array.

To make it possible to distinguish between hyperlinks to the different parents, we can add additional properties like "hasPersonRelationshipType"

Disclaimer: Simplified examples are not CEDS-aligned.

Classes in JSON-LD Documents vs Tables in the IDS



Earlier in this training, we described 'classes' as being like 'tables' because both classes and tables have properties (fields) used to store data about an entity (a person, organization, course, etc.)

In the IDS, the data about an entity is spread across many records in many different fact tables. One table can contain data on multiple entities.

JSON-LD flips that paradigm. Each JSON-LD document holds data for one entity, but that data may be from many different classes.

STUDENTS		
STUID	SNAME	SINFO
S3245	Jill	...
S8524	John	...
S1735	Jane	...
S3409	Jim	...



A Single JSON Document

```
{
  student : "Jill",
  schedule :
    [ {
      time : "14:00",
      course : "Math 101",
      room : "A102",
      teacher : "Adam"
    },
    {
      time : "16:00",
      course : "Science 102",
      room : "B405",
      teacher : "Anita"
    }
  ]
}
```

Disclaimer: Simplified examples are not CEDS-aligned.

Or... another way to describe this...



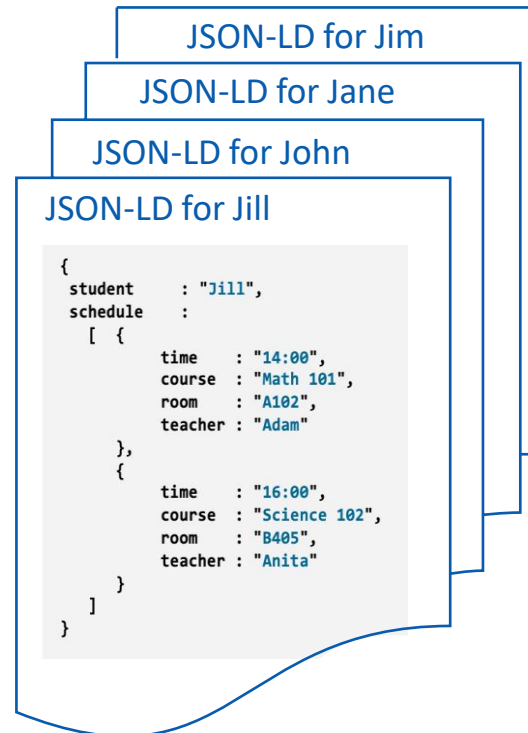
When storing data a relational database (like the IDS), a person browsing the database will see a **collection of tables**.

TEACHERS		
COURSES		
SCHEDULES		
STUDENTS		
STUID	SNAME	SINFO
S3245	Jill	...
S8524	John	...
S1735	Jane	...
S3409	Jim	...

Each table contains *all* the data related to a single concept (a.k.a. class) – like *teachers*, *courses*, or *schedules*.

Each record in that table holds a small piece of that conceptual data for a specific entity (*Jill*, *John*, *Jane* or *Jim*)

When storing data in JSON-LD documents in something like a NoSQL database, a person browsing the database will literally see a **collection of JSON-LD documents**.



Each document holds data exclusively related to a specific entity (*Jill*, *John*, *Jane* or *Jim*)

There is no document listing all the teachers. No document with all schedules in one place. Instead, that conceptual data are broken up and embedded in each student's individual JSON-LD document.

How does storing data in JSON-LD compare to storing data in the IDS?



We can store CEDS-aligned transactional data in either the IDS or the JSON-LD:

- Both provide ways to link data
- Both use normalized structures to make writing and updating data faster.

However, many organization are switching to prefer JSON-LD over the IDS for CEDS transactional data because

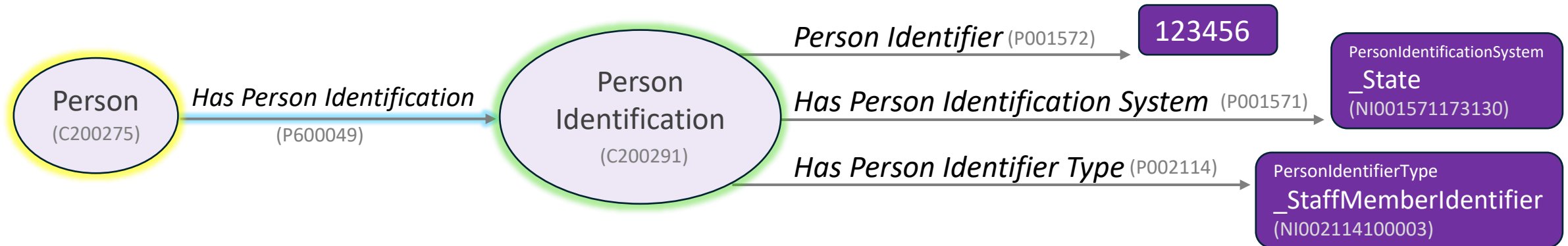
- JSON is more ubiquitous for moving data – so it's easier to access and move the data across different applications and different development languages.
- JSON-LD has links to the CEDS ontology embedded right inside it – making it easier for downstream systems to access all that rich contextual information. The ontology also helps machine-learning applications (or AI) build knowledge graphs that can be used to improve its understanding of the relationships among the data.
- JSON-LD is more flexible than the IDS – it's better at handling optional fields, nested structures and variable data than the IDS. For example, new elements can be added to JSON-LD without having to change an entire database schema.
- JSON-LD is ideal for APIs and real-time exchange of data like event driven architecture or microservices.

The previous JSON-LD examples said they were simplified for training purposes and not based on CEDS.



So, what does JSON-LD look like when it is based on the CEDS Ontology?

Creating JSON-LD based on the CEDS Ontology



Let's start with a familiar example from earlier: storing a staff member's identifier as part of their person record.

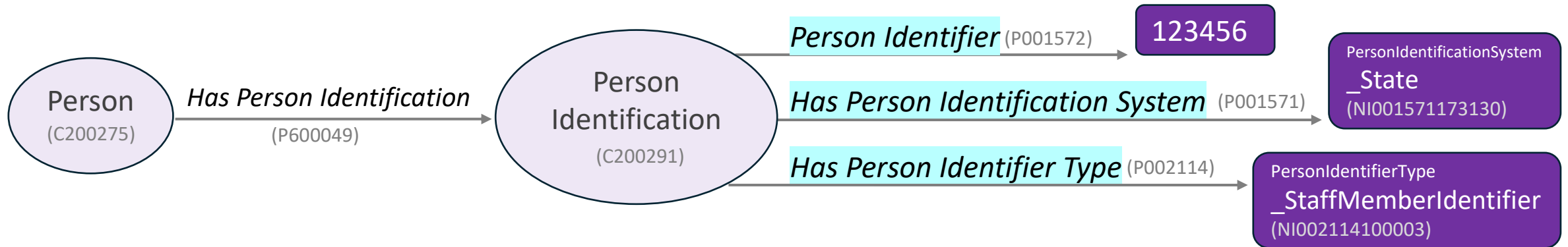
The highest level class for this document is "Person" so that is going to be the **@type** at the top.

The **key-value** pair is going to be the object property 'Has Person Identification' as the key and the class 'Person Identification' as the value.

```
{
  "@id": "https://www.namespace.example/person/578964532" ,
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",
  "hasPersonIdentification":
    { "@type" : "PersonIdentification"
    }
}
```

We're going to put this particular value in curly brackets because this class isn't the final destination. The class is just a pit stop on the way to the properties that hold the actual data. The curly brackets will help us group everything that is going to be a part of PersonIdentification.

Nesting properties inside a class



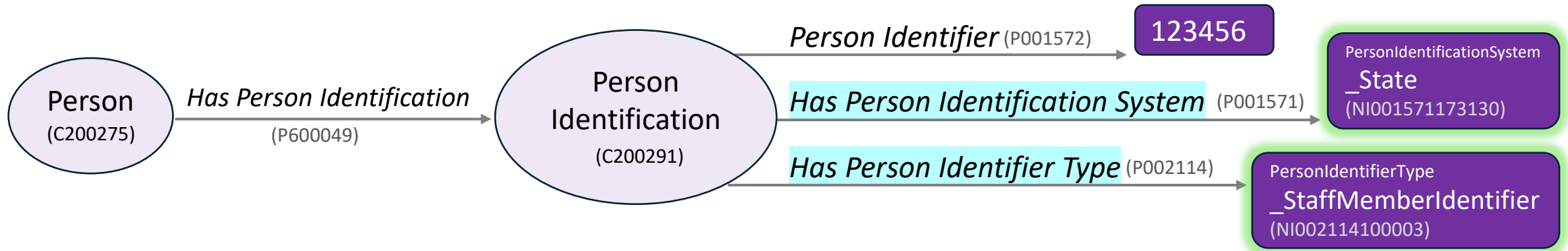
Once we're inside the Person Identification class, we have more properties, and so that means we're going to have more key-value pairs inside the block of JSON representing this class.

The properties are the **keys**.

PersonIdentifier is a data property that stores an alphanumeric identifier – which is what we'll store as the **value** for that pair.

```
{
  "@id": "https://www.namespace.example/person/578964532",
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",
  "hasPersonIdentification": {
    "@type": "PersonIdentification",
    "PersonIdentifier": "123456",
    "hasPersonIdentificationSystem": _____,
    "hasPersonIdentifierType": _____
  }
}
```

Populating the value from concept schemes



The next two **keys** are object properties pointing to a specific option in a concept scheme.

Their **values** will be represented by using the **concept scheme name**, an **underscore**, and then the **option code**.

Now we've clearly captured that this person has "123456" as their state-assigned staff member identifier.

But what about their other identifiers?

```
{
  "@id": "https://www.namespace.example/person/578964532",
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",
  "hasPersonIdentification":
    {
      "@type": "PersonIdentification",
      "PersonIdentifier": "123456",
      "hasPersonIdentificationSystem":
        "PersonIdentificationSystem_State",
      "hasPersonIdentifierType":
        "PersonIdentifierType_StaffMemberIdentifier"
    }
}
```

Using the power of arrays

Most people have more than one identifier, so what we really want is an **array** showing all the identifiers this person has.

The array itself is defined by square brackets `[]`.

Since each element in the array is a class + properties, we're going to put curly brackets `{ }` around each element and commas between them.

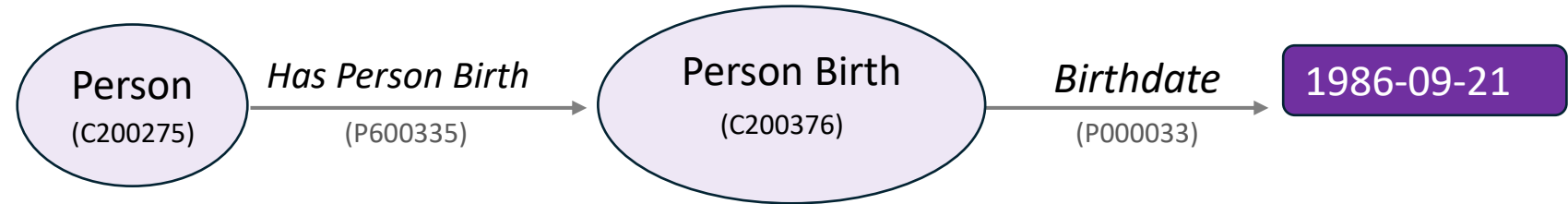
Now we can capture other identifiers – like their district-assigned student identifier and their current driver's license number.

```
{
  "@id": "https://www.namespace.example/person/578964532",
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",
  "hasPersonIdentification": [
    {
      "@type": "PersonIdentification",
      "PersonIdentifier" : "123456",
      "hasPersonIdentificationSystem" :
        "PersonIdentificationSystem_State",
      "hasPersonIdentifierType" :
        "PersonIdentifierType_StaffMemberIdentifier"
    },
    {
      "@type": "PersonIdentification",
      "PersonIdentifier" : "333221",
      "hasPersonIdentificationSystem" :
        "PersonIdentificationSystem_District",
      "hasPersonIdentifierType" :
        "PersonIdentifierType_StudentIdentifier"
    },
    {
      "@type": "PersonIdentification",
      "PersonIdentifier" : "S 000 123 456 789",
      "hasPersonIdentificationSystem" :
        "PersonIdentificationSystem_DriversLicense",
      "hasPersonIdentifierType" :
        "PersonIdentifierType_PersonIdentifier"
    }
  ]
}
```

Populating values that are not strings



So far, our CEDS-aligned JSON-LD examples have only had values that were strings*.



Values that are strings are simply enclosed in quotation marks. Values that are integers or decimals are not required to have quotation marks around them unless you want to restrict their values in some way.

Every other value should have its datatype explicitly declared using the `@type` tag *and* be enclosed in quotes.

If the value is an entire class, then the `@type` will name the class.

If the value is a date, datetime, special decimal, etc. then the `@type` will identify the XML datatype.

Note that we're using 'xsd' as an alias for the XML namespace.

```
"hasPersonBirth": {
  "@type" : "PersonBirth",
  "Birthdate" : {
    "@value" : "1986-09-21",
    "@type" : "xsd:date"
  }
}
```

** PersonIdentifier has a datatype of 'token' which is a type of string. So even identifiers that look like a number are actually stored as a string.*

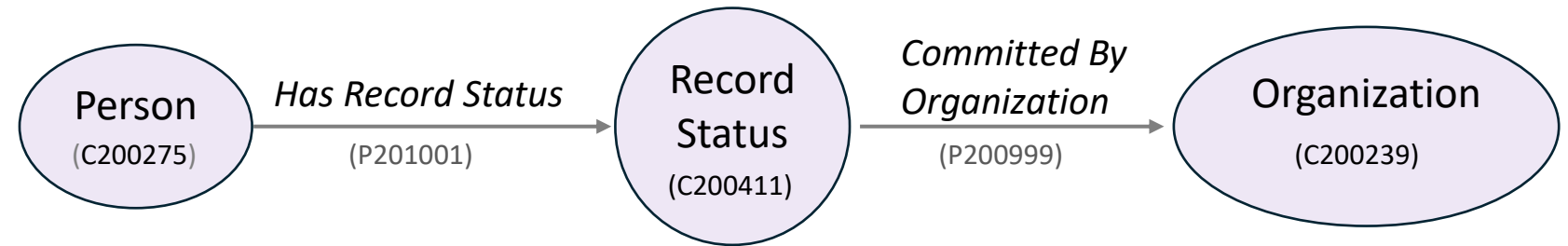
Populating values that are links to other resources



Let's say that you wanted to document which organization submitted the data about this person. You can do that with the 'Committed By Organization' property.

But instead of storing the organization's name, you want to link to that organization's JSON-LD document – so that you can access the power of linked data!

To link to other resources, use an @id field with the IRI (unique identifier) pointing to the destination resource (that organization's JSON-LD).



```
{
  "@id": "https://www.namespace.example/person/578964532",
  "@type": "Person",
  "@context": "https://www.namespace.example/context.json",

  "hasRecordStatus": {
    "@type": "RecordStatus",
    "Committed By Organization": {
      "@id": "https://www.namespace.example/organization/g57rpd",
      "@type": "Organization"
    }
  }
}
```

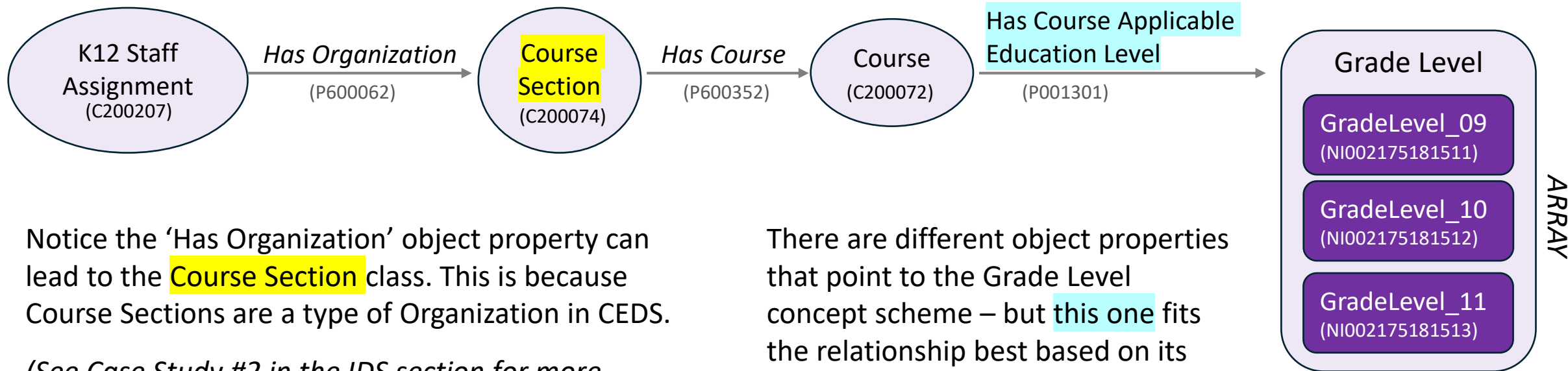
The @type field identifies the class of the resource you are linking to.

Sometimes the path can be more complex



Suppose that we want to store data about a high school teaching assignment in a JSON-LD document. We might want to store which grade levels are associated with that assignment.

This graph shows us the path through the CEDS Ontology from the K12 Staff Assignment class to options in the Grade Level concept scheme.

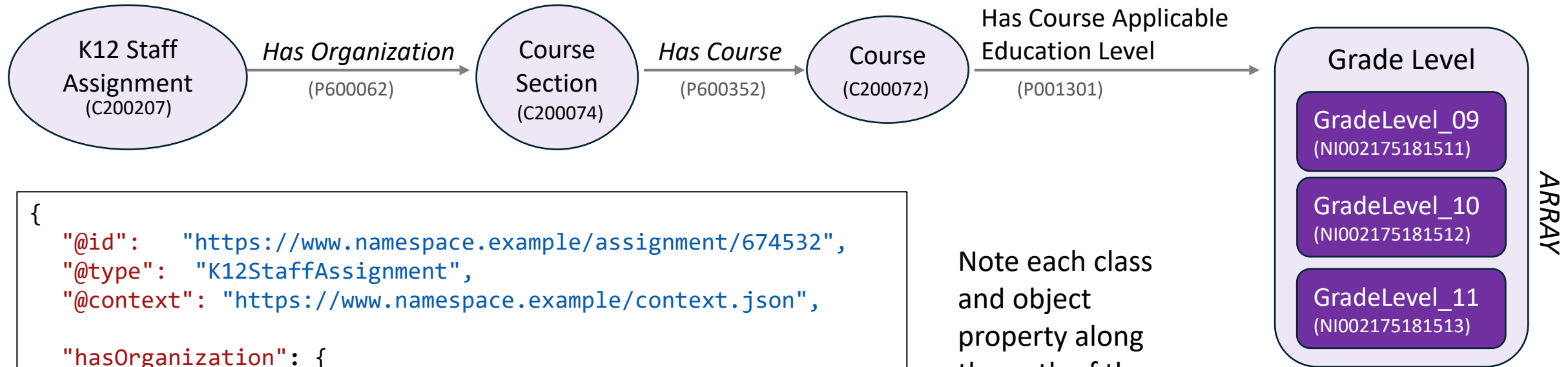


Notice the 'Has Organization' object property can lead to the **Course Section** class. This is because Course Sections are a type of Organization in CEDS.

(See Case Study #2 in the IDS section for more details about this)

There are different object properties that point to the Grade Level concept scheme – but **this one** fits the relationship best based on its definition *(See slide 47 for other properties leading to Grade Level)*

Sometimes there is quite a bit of nesting



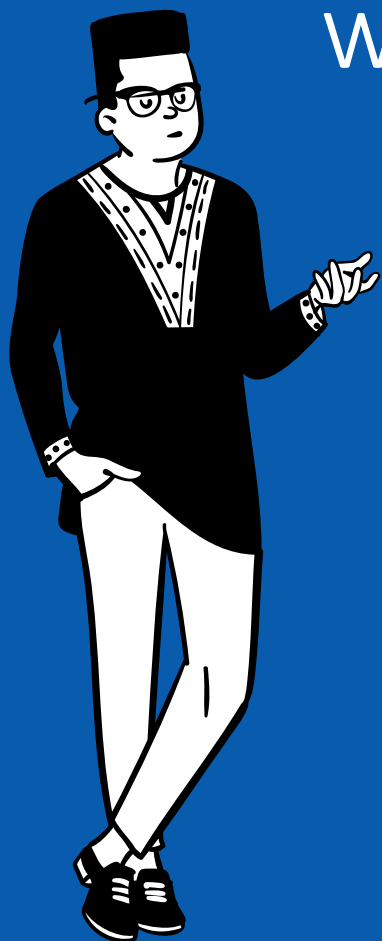
```
{
  "@id": "https://www.namespace.example/assignment/674532",
  "@type": "K12StaffAssignment",
  "@context": "https://www.namespace.example/context.json",

  "hasOrganization": {
    "@type" : "CourseSection",
    "hasCourse" : {
      "@type" : "Course",
      "hasCourseApplicableEducationLevel" : [
        "GradeLevel_09",
        "GradeLevel_10",
        "GradeLevel_11"
      ]
    }
  }
}
```

Note each class and object property along the path of the graph is part of the JSON-LD document. The child class nodes are nested inside of the parents.

Also note that when we store an **array of strings** (like with option set values), then we don't need the curly brackets around each element in the array.

Why does the ontology matter when we're storing data in JSON-LD?



The CEDS and CEPI ontology documents define relationships between elements – which means they are also defining how the data are structured in the JSON-LD.

Understanding how the structure of the ontology impacts the structure of the JSON-LD will be important for analysts who are

- Performing data quality checks on transactional data
- Helping with to develop new JSON-LD
- Helping to develop or maintain SHACL

Context is King – Part 2



These two snippets of JSON-LD are functionally identical.

The top one uses the globally unique identifiers (IRI's) for most of the keys and values. This is what applications need to function with speed and precision, and what AI needs to be able to contextualize data.

WHAT APPLICATIONS SEE IN THE JSON-LD

```
"https://w3id.org/CEDStandards/terms/P600335": {  
  "@type" : "https://w3id.org/CEDStandards/terms/C200376",  
  "https://w3id.org/CEDStandards/terms/P000033": {  
    "@value" : "1986-09-21",  
    "@type" : "http://www.w3.org/2001/XMLSchema#date"  
  }  
}
```

For humans though? That top one is pretty much unintelligible.

Luckily CEDS has a context file that makes it possible to translate back and forth between the human- and machine- readable versions of a JSON-LD document as needed. This enables us to develop using the human-readable version and then store the transactional data in the faster, machine-readable version.

WHAT HUMANS SEE IN THE JSON-LD

```
"hasPersonBirth": {  
  "@type" : "PersonBirth",  
  "Birthdate" : {  
    "@value" : "1986-09-21",  
    "@type" : "xsd:date"  
  }  
}
```

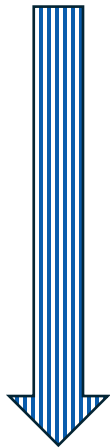
Aliases for more than just namespaces



Namespace
aliases

```
1  {
2    "@context": {
3      "@vocab": "https://w3id.org/CEDStandards/terms/",
4      "@base": "https://w3id.org/CEDStandards/terms/",
5      "rdf": "http://www.w3.org/1999/02/22-rdf-syntax-ns#",
6      "rdfs": "http://www.w3.org/2000/01/rdf-schema#",
7      "xsd": "http://www.w3.org/2001/XMLSchema#",
8      "owl": "http://www.w3.org/2002/07/owl#",
9      "skos": "http://www.w3.org/2004/02/skos/core#",
10     "dc": "http://purl.org/dc/elements/1.1/",
11     "ceds": "https://w3id.org/CEDStandards/terms/",
12     "schema": "http://schema.org/",
13     "CEDSBaseResource": "ceds:C000000",
14     "RecordStatus": "ceds:C200411",
15     "RecordStatusHistory": "ceds:C200395",
16     "DataCollection": "ceds:C200410",
17     "AccreditingOrganization": "ceds:C200393",
18     "Activity": "ceds:C200001",
19     "ActivityRecognition": "ceds:C200002",
20     "AdultEducationCourse": "ceds:C200003",
21     "AdultEducationProvider": "ceds:C200004",
22     "AdultEducationStaff": "ceds:C200005",
```

Class,
property,
and option
aliases



Earlier in the training, we talked about how the context file would help connect the data in JSON-LD documents to the ontologies and namespaces being referenced – thereby providing valuable context to both humans and AI.

However, the CEDS context file goes one step further by providing human-friendly aliases for *every* identifier associated with a CEDS class, property, or option.

Fun fact: Did you notice that the context file is formatted as JSON?

How do the aliases work?



AN EXCERPT from a JSON-LD document:
(i.e. what humans see)

```
"hasPersonBirth": {  
  "@type" : "PersonBirth",  
  "Birthdate" : {  
    "@value" : "1986-09-21",  
    "@type" : "xsd:date"  
  }  
}
```

EXCERPTS from the context file:

```
"ceds": "https://w3id.org/CEDStandards/terms/",  
"xsd": "http://www.w3.org/2001/XMLSchema#",  
"Birthdate": "ceds:P000033",  
"hasPersonBirth": "ceds:P600335",  
"PersonBirth": "ceds:C200376",
```

The application uses the context file to translate the human-readable JSON-LD document into something more explicitly machine-readable

"hasPersonBirth" becomes "ceds:P600335"

But the 'ceds' part is also an alias so it gets translated again:

"ceds:P600335" then becomes

"<https://w3id.org/CEDStandards/terms/P600335>"

"Birthdate" is similarly translated into

"<https://w3id.org/CEDStandards/terms/P000033>"

And "xsd:date" becomes

"<http://www.w3.org/2001/XMLSchema#date>"

And so on.

Time to Reflect & Digest on JSON-LD



- What is JSON-LD and why would we use it? Slides 72-73, 80
- How does JSON-LD compare to the IDS? Slides 78-80
- Review JSON-LD structures and conventions:
 - What are key-value pairs? Slides 72, 82-84
 - How is the @id field used? Slides 74-75, 87
 - How is the @type field used? Slides 74-75, 82, 86-87
 - How does the @context field work? Slides 74, 76,
 - How do arrays work? Slides 77, 85, 89
 - How are option sets represented in JSON? Slides 84-85, 88-89
 - How does the ontology influence how we structure key-value pairs and how we might end up nesting them? Slides 82-84, 88-89
- What is the purpose of the context file? Why is it so important? Slides 91-93
- REFLECT:
 - How might these structures and conventions impact your work? What questions do you have?

CLOSING

This training was designed to enable you to:

- Describe the benefits of using the CEDS IDS, the CEDS Ontology, and JSON-LD.
- Understand some of the most common structures and conventions associated with the CEDS IDS, the CEDS Ontology, and JSON-LD.
- Think about how the structures and conventions of the CEDS IDS, the CEDS Ontology, and JSON-LD might impact your work as an analyst.

Feedback on this training is welcome! Questions, corrections and suggestions can be directed to ceds@aemcorp.com.

Additional CEDS Resources

For deep-dives into CEDS modeling:

- [CEDS Data Model Guide](#) (v.14 2026 – focuses more on IDS)
- [CEDS Ontology Model](#) (v.14 2026)

Everything in CEDS is open-source and shared on GitHub. To review related code, documentation, or to participate in the development of the next version of CEDS, visit:

- [CEDS-IDS](#)
- [CEDS-Ontology](#)

The CEDS Collaborative Exchange on GitHub has a [CEDS-SHACL-JSON-LD-Framework](#) repository that holds the CEDS context file and other resources to help with implementing the CEDS ontology in JSON-LD.

For more resources about CEDS in general or training on the CEDS ontology in particular, see the [CEDS Gitbook](#).

Additional Technical Resources

Curious about a technical topics mentioned in the training? Check out these links to learn more

3rd Normal Form / Database Normalization

- ["Normalization in Relational Databases: First Normal Form \(1NF\), Second Normal Form \(2NF\), and Third Normal Form \(3NF\)"](#) , Redgate
- ["Normal Form in DBMS"](#) , GeeksForGeeks

Ontologies

- ["489:Unlock Ontology: Computer Science Definition Explained"](#) , PowerBITips, YouTube video (90 sec)
- ["Introduction to Ontologies"](#) , GeeksForGeeks, 2026-05-23
- ["What is an Ontology? Definition & Types"](#) by Hao Wu on PuppyGraph, 2026-05-01

JSON-LD

- [JSON-LD.org](#) (official website)
- ["Paul's notes on how JSON-LD works"](#) by Paul Frazee on Medium, 2022-05-04
- ["Beginner's Guide to JSON-LD"](#) by Dillon Redding on Medium, 2021-11-18